




# Probabilistic model checking (part 2)

Marta Kwiatkowska

Department of Computer Science, University of Oxford

MPP Model Checking Workshop, 26-27 February 2015



# Part 2

Markov decision processes

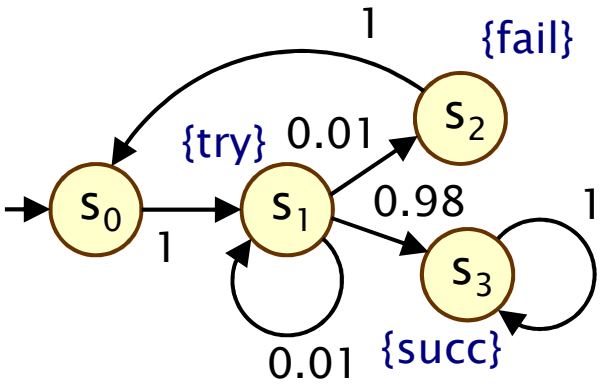


# Overview (Part 2)

- Introduction
- Model checking for Markov decision processes (MDPs)
  - MDPs: definition
  - Paths, strategies & probability spaces
  - PCTL model checking
- Strategy synthesis for MDPs
  - Properties and objectives
  - Verification vs synthesis
  - Case study: Dynamic power management
- Summary and further work

# Recap: Discrete-time Markov chains

- Discrete-time Markov chains (DTMCs)
  - state-transition systems augmented with probabilities
- Formally: DTMC $D = (S, s_{init}, P, L)$ where:
  - $S$ is a set of states and $s_{init} \in S$ is the initial state
  - $P : S \times S \rightarrow [0,1]$ is the transition probability matrix
  - $L : S \rightarrow 2^{AP}$ labels states with atomic propositions
  - define a probability space $Pr_s$ over paths $Path_s$




- Properties of DTMCs
  - can be captured by the logic PCTL
  - e.g. send → $P_{\geq 0.95}$ [ F deliver ]
  - key question: what is the probability of reaching states $T \subseteq S$ from state $s$?
  - reduces to graph analysis + linear equation system



# Nondeterminism

- Some aspects of a system may not be probabilistic and should not be modelled probabilistically; for example:

- Concurrency – scheduling of parallel components
  - e.g. randomised distributed algorithms – multiple probabilistic processes operating asynchronously

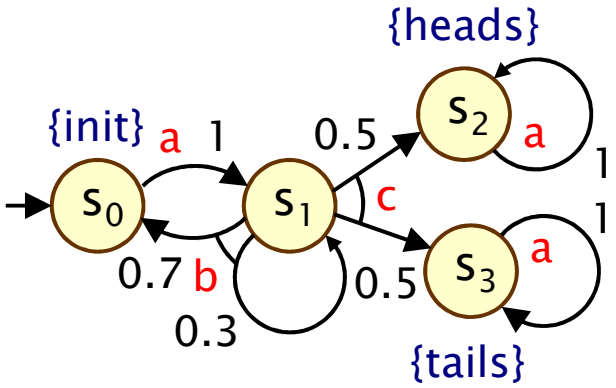
- Underspecification – unknown model parameters
  - e.g. a probabilistic communication protocol designed for message propagation delays of between $d_{min}$ and $d_{max}$

- Unknown environments
  - e.g. probabilistic security protocols – unknown adversary



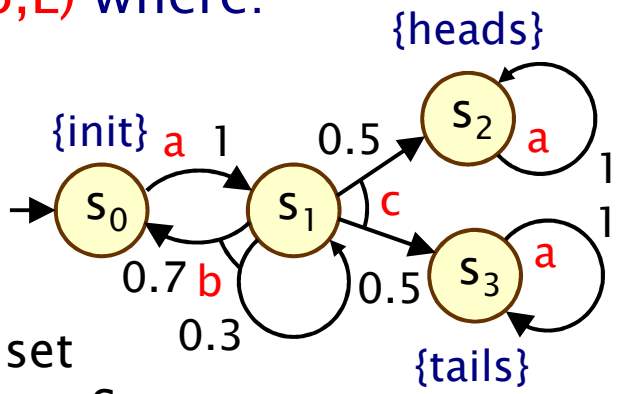
# Markov decision processes

- Markov decision processes (MDPs)
  - extension of DTMCs which allow nondeterministic choice
- Like DTMCs:
  - discrete set of states representing possible configurations of the system being modelled
  - transitions between states occur in discrete time-steps
- Probabilities and nondeterminism
  - in each state, a nondeterministic choice between several discrete probability distributions over successor states



# Markov decision processes

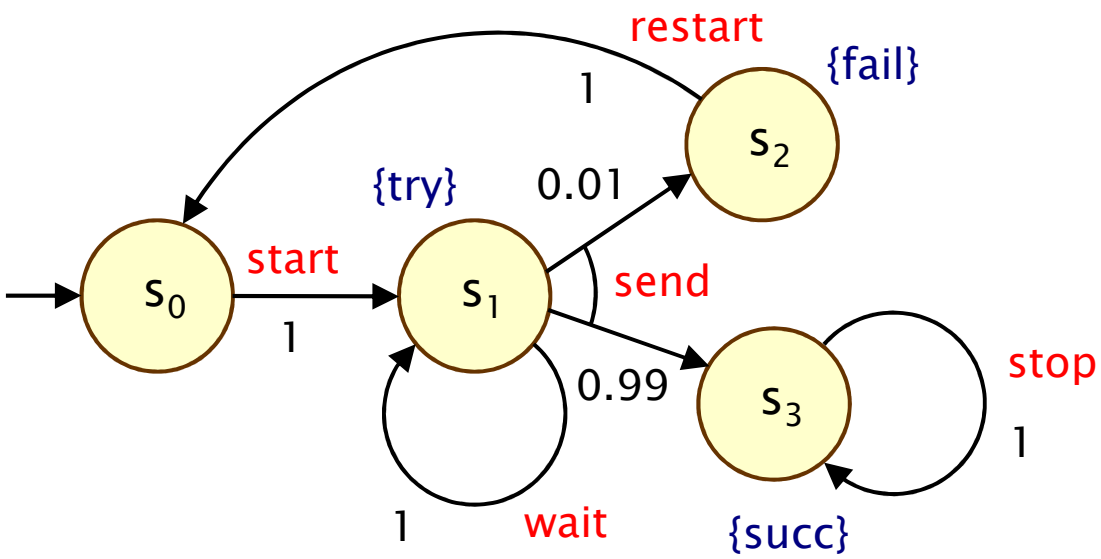
- Formally, an MDP M is a tuple $(S, s_{init}, \alpha, \delta, L)$ where:
  - $S$ is a set of states ("state space")
  - $s_{init} \in S$ is the initial state
  - $\alpha$ is an alphabet of action labels
  - $\delta \subseteq S \times \alpha \times Dist(S)$ is the transition probability relation, where Dist(S) is the set of all discrete probability distributions over S
  - $L : S \rightarrow 2^{AP}$ is a labelling with atomic propositions




- Notes:
  - we also abuse notation and use $\delta$ as a function
  - i.e. $\delta : S \rightarrow 2^{\alpha \times Dist(S)}$ where $\delta(s) = \{ (a,\mu) \mid (s,a,\mu) \in \delta \}$
  - we assume $\delta(s)$ is always non-empty, i.e. no deadlocks
  - MDPs, here, are identical to probabilistic automata [Segala]
    - usually, MDPs take the form: $\delta : S \times \alpha \rightarrow Dist(S)$

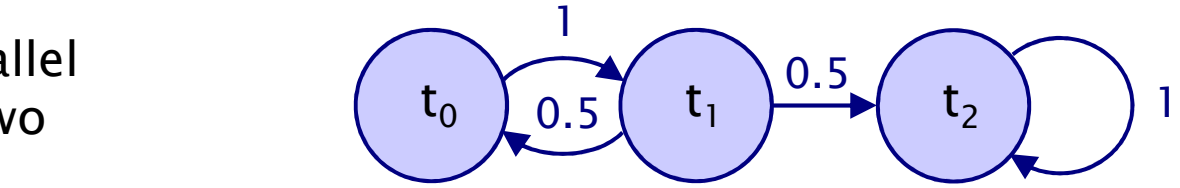
# Simple MDP example

- A simple communication protocol
  - after one step, process starts trying to send a message
  - then, a nondeterministic choice between: (a) waiting a step because the channel is unready; (b) sending the message
  - if the latter, with probability 0.99 send successfully and stop
  - and with probability 0.01, message sending fails, restart

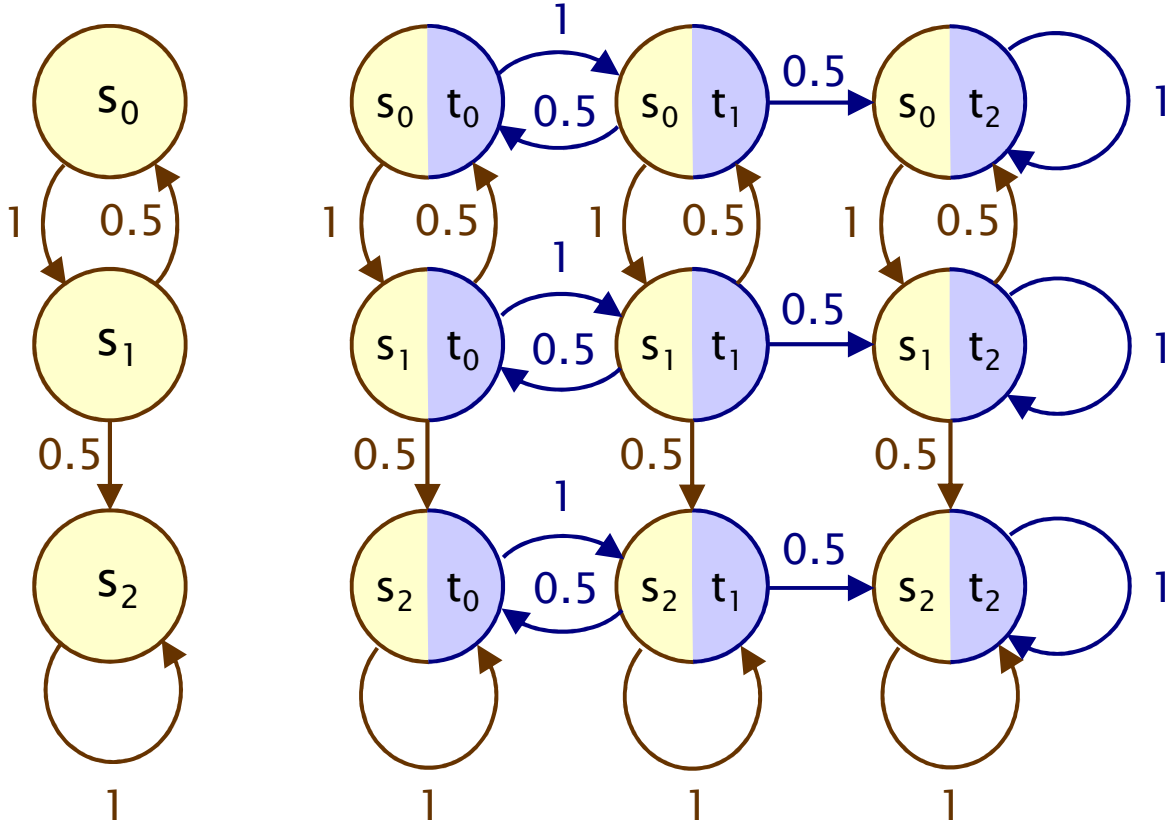


# Example – Parallel composition

Asynchronous parallel composition of two 3-state DTMCs

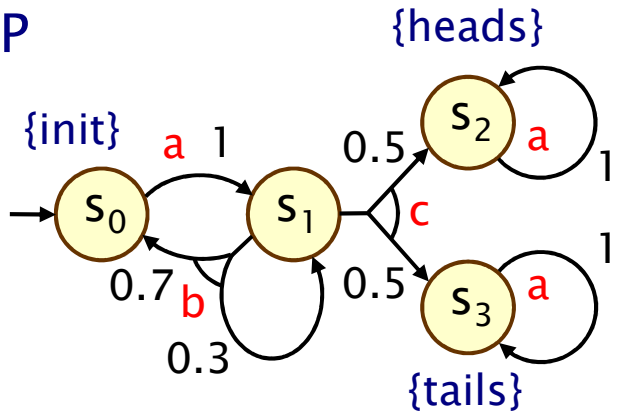



Action labels omitted here



# Paths and strategies

- A (finite or infinite) path through an MDP
  - is a sequence $(s_0 \ldots s_n)$ of (connected) states
  - represents an execution of the system
  - resolves both the probabilistic and nondeterministic choices




- A strategy σ (aka. "adversary" or "policy") of an MDP
  - is a resolution of nondeterminism only
  - is (formally) a mapping from finite paths to distributions on action-distribution pairs
  - induces a fully probabilistic model
  - i.e. an (infinite-state) Markov chain over finite paths
  - on which we can define a probability space over infinite paths



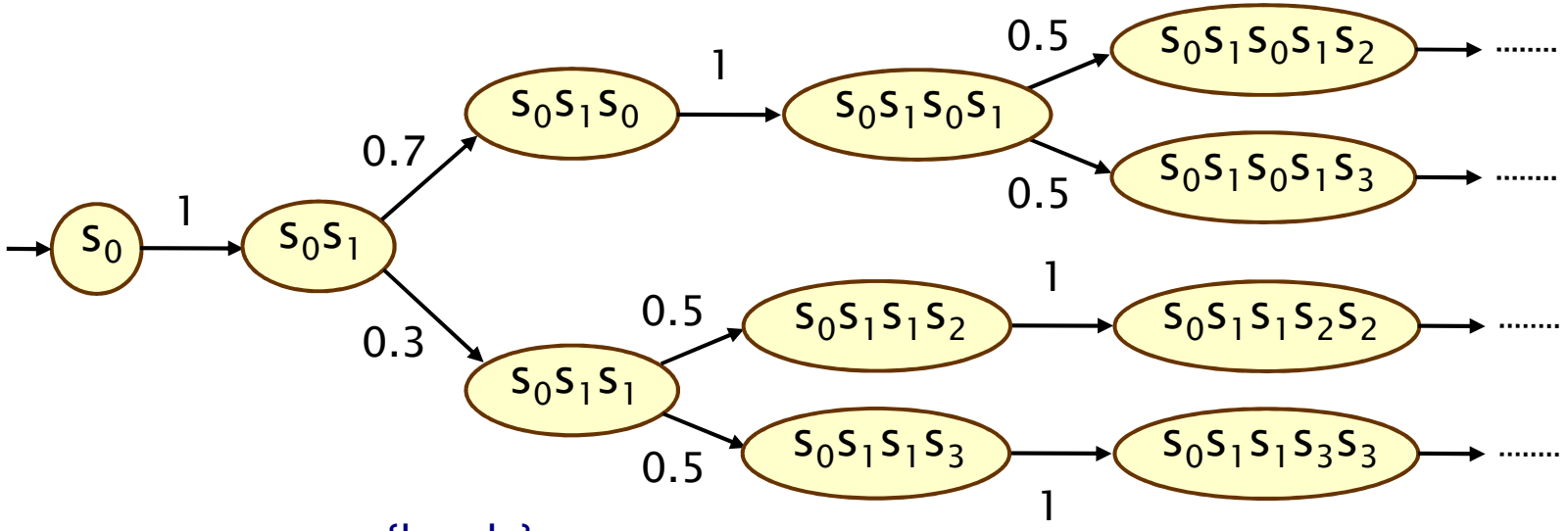
# Classification of strategies

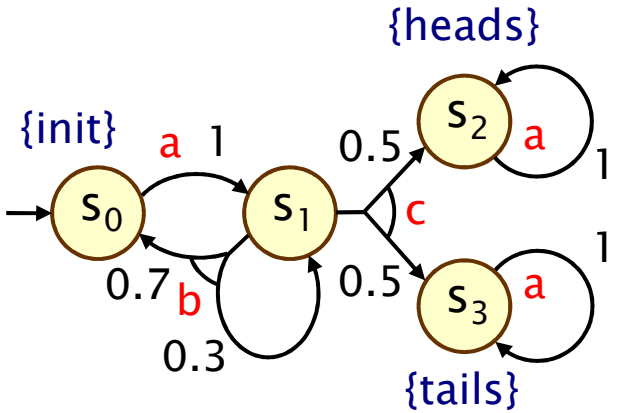
- Strategies are classified according to
- randomisation:
  - $\sigma$ is deterministic (pure) if $\sigma(s_0 \ldots s_n)$ is a point distribution, and randomised otherwise
- memory:
  - $\sigma$ is memoryless (simple) if $\sigma(s_0 \ldots s_n) = \sigma(s_n)$ for all $s_0 \ldots s_n$
  - $\sigma$ is finite memory if there are finitely many modes such as $\sigma(s_0 \ldots s_n)$ depends only on $s_n$ and the current mode, which is updated each time an action is performed
  - otherwise, $\sigma$ is infinite memory

- A strategy $\sigma$ induces, for each state $s$ in the MDP:
  - a set of infinite paths $\text{Path}^\sigma(s)$
  - a probability space $\Pr^\sigma_s$ over $\text{Path}^\sigma(s)$

# Example strategy

- Fragment of induced Markov chain for strategy which picks b then c in $s_1$




finite-memory, deterministic



# PCTL

- Temporal logic for properties of MDPs (and DTMCs)
  - extension of (non-probabilistic) temporal logic CTL
  - key addition is probabilistic operator P
  - quantitative extension of CTL's A and E operators

- PCTL syntax:
  - $\phi$ ::= true | a | $\phi \wedge \phi$ | $\neg\phi$ | $P_{\sim p}$ [ $\psi$ ] (state formulas)
  - $\psi$ ::= $X\,\phi$ | $\phi\ U^{\leq k}\ \phi$ | $\phi\ U\ \phi$ (path formulas)
  - where a is an atomic proposition, used to identify states of interest, $p \in [0,1]$ is a probability, $\sim \in \{<,>,\leq,\geq\}$, $k \in \mathbb{N}$

  - Example: send → $P_{\geq 0.95}$ [ true $U^{\leq 10}$ deliver ]

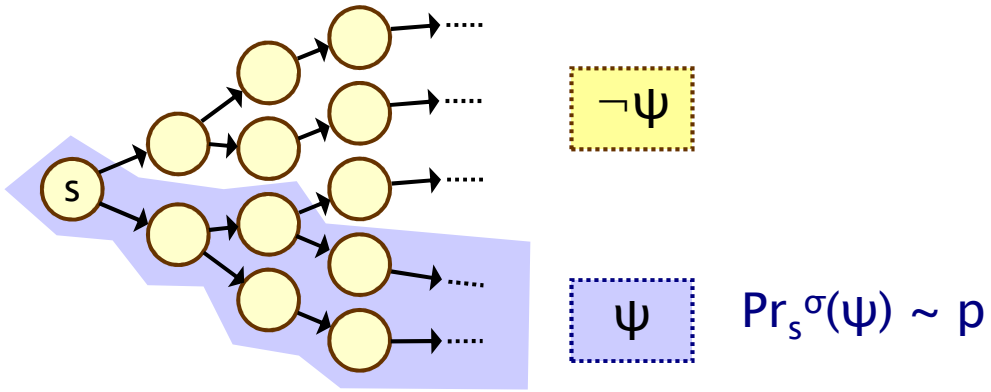


# PCTL semantics for MDPs

- PCTL formulas interpreted over states of an MDP
  - $s \models \phi$ denotes $\phi$ is "true in state s" or "satisfied in state s"
- Semantics of (non-probabilistic) state formulas:
  - for a state s of the MDP $(S, s_{init}, \alpha, \delta, L)$:
  - $s \models a \quad \Leftrightarrow \quad a \in L(s)$
  - $s \models \phi_1 \wedge \phi_2 \quad \Leftrightarrow \quad s \models \phi_1$ and $s \models \phi_2$
  - $s \models \neg\phi \quad \Leftrightarrow \quad s \models \phi$ is false
- Semantics of path formulas:
  - for a path $\omega = s_0(a_0,\mu_0)s_1(a_1,\mu_1)s_2\ldots$ in the MDP:
  - $\omega \models X\,\phi \quad \Leftrightarrow \quad s_1 \models \phi$
  - $\omega \models \phi_1\, U^{\leq k}\, \phi_2 \quad \Leftrightarrow \quad \exists i \leq k$ such that $s_i \models \phi_2$ and $\forall j < i$, $s_j \models \phi_1$
  - $\omega \models \phi_1\, U\, \phi_2 \quad \Leftrightarrow \quad \exists k \geq 0$ such that $\omega \models \phi_1\, U^{\leq k}\, \phi_2$

# PCTL semantics for MDPs

- Semantics of the probabilistic operator P
  - can only define probabilities for a specific strategy σ
  - $s \models P_{\sim p}[\psi]$ means "the probability, from state s, that ψ is true for an outgoing path satisfies ~p for all strategies σ"
  - formally $s \models P_{\sim p}[\psi] \Leftrightarrow Pr_s^{\sigma}(\psi) \sim p$ for all strategies σ
  - where we use $Pr_s^{\sigma}(\psi)$ to denote $Pr_s^{\sigma}\{\omega \in Path_s^{\sigma} \mid \omega \models \psi\}$




- Some equivalences:
  - $F\,\phi \equiv \Diamond\,\phi \equiv true\ U\ \phi$ (eventually, "future")
  - $G\,\phi \equiv \Box\,\phi \equiv \neg(F\,\neg\phi)$ (always, "globally")



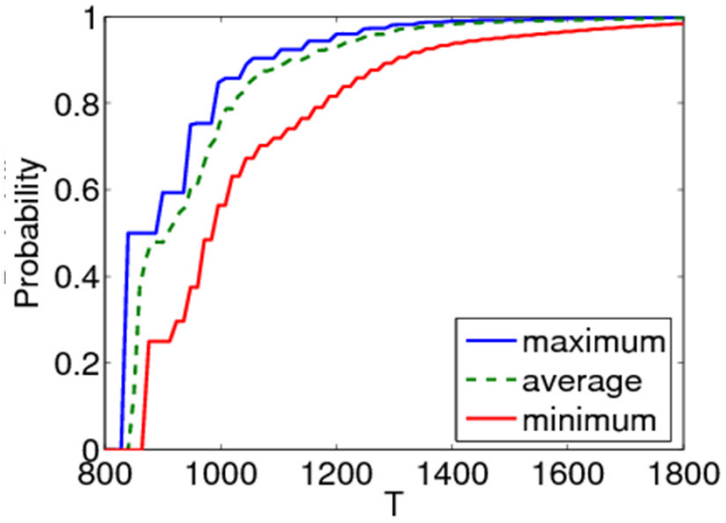
# Minimum and maximum probabilities

- Letting:
  - $Pr_s^{max}(\psi) = \sup_\sigma Pr_s^\sigma(\psi)$
  - $Pr_s^{min}(\psi) = \inf_\sigma Pr_s^\sigma(\psi)$
- We have:
  - if $\sim \in \{\geq, >\}$, then $s \models P_{\sim p}[\ \psi\ ] \Leftrightarrow Pr_s^{min}(\psi) \sim p$
  - if $\sim \in \{<, \leq\}$, then $s \models P_{\sim p}[\ \psi\ ] \Leftrightarrow Pr_s^{max}(\psi) \sim p$
- Model checking $P_{\sim p}[\ \psi\ ]$ reduces to the computation over all strategies of either:
  - the minimum probability of $\psi$ holding
  - the maximum probability of $\psi$ holding
- Crucial result for model checking PCTL on MDPs
  - memoryless strategies suffice, i.e. there are always memoryless strategies $\sigma_{min}$ and $\sigma_{max}$ for which:
  - $Pr_s^{\sigma_{min}}(\psi) = Pr_s^{min}(\psi)$ and $Pr_s^{\sigma_{max}}(\psi) = Pr_s^{min}(\psi)$

# Quantitative properties

- For PCTL properties with P as the outermost operator
  - quantitative form (two types): $P_{min=?}$ [ ψ ] and $P_{max=?}$ [ ψ ]
  - i.e. "what is the minimum/maximum probability (over all adversaries) that path formula ψ is true?"
  - corresponds to an analysis of best-case or worst-case behaviour of the system
  - model checking is no harder since compute the values of $Pr_s^{min}(\psi)$ or $Pr_s^{max}(\psi)$ anyway
  - useful to spot patterns/trends

- Example: CSMA/CD protocol
  - "min/max probability that a message is sent within the deadline"





# PCTL model checking for MDPs

- Algorithm for PCTL model checking [BdA95]
  - inputs: MDP $M=(S,s_{init},\alpha,\delta,L)$, PCTL formula $\phi$
  - output: $Sat(\phi) = \{ s \in S \mid s \models \phi \}$ = set of states satisfying $\phi$
- Basic algorithm same as PCTL model checking for DTMCs
  - proceeds by induction on parse tree of $\phi$
  - non-probabilistic operators (true, a, $\neg$, $\wedge$) straightforward
- Only need to consider $P_{\sim p}[\ \psi\ ]$ formulas
  - reduces to computation of $Pr_s^{min}(\psi)$ or $Pr_s^{max}(\psi)$ for all $s \in S$
  - dependent on whether $\sim \in \{\geq,>\}$ or $\sim \in \{<,\leq\}$
  - these slides cover the case $Pr_s^{min}(\phi_1\ U\ \phi_2)$, i.e. $\sim \in \{\geq,>\}$
  - case for maximum probabilities is very similar
  - next ($X\ \phi$) and bounded until ($\phi_1\ U^{\leq k}\ \phi_2$) are straightforward extensions of the DTMC case

# PCTL until for MDPs

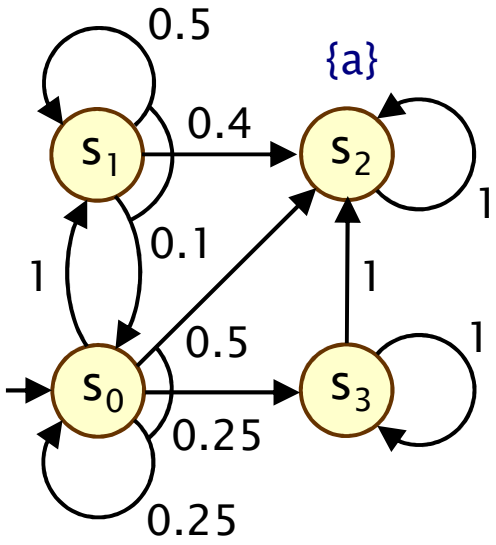
- Computation of probabilities $Pr_s^{min}(\phi_1 \, U \, \phi_2)$ for all $s \in S$
- First identify all states where the probability is 1 or 0
  - "precomputation" algorithms, yielding sets $S^{yes}$, $S^{no}$
- Then compute (min) probabilities for remaining states ($S^?$)
  - either: solve linear programming problem
  - or: approximate with an iterative solution method
  - or: use policy iteration

Example:

$P_{\geq p}$ [ F a ]

≡

$P_{\geq p}$ [ true U a ]

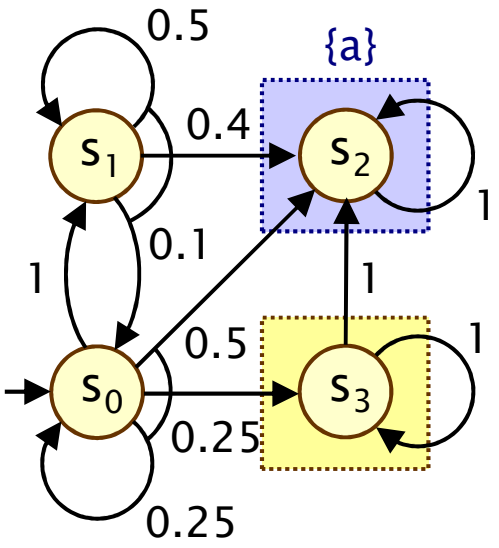


# PCTL until – Precomputation

- Identify all states where $Pr_s^{min}(\phi_1 \; U \; \phi_2)$ is 1 or 0
  - $S^{yes} = Sat(P_{\geq 1} [ \phi_1 \; U \; \phi_2 ])$, $S^{no} = Sat(\neg P_{>0} [ \phi_1 \; U \; \phi_2 ])$
- Two graph-based precomputation algorithms:
  - algorithm Prob1A computes $S^{yes}$
    - for all strategies the probability of satisfying $\phi_1 \; U \; \phi_2$ is 1
  - algorithm Prob0E computes $S^{no}$
    - there exists a strategy for which the probability is 0

Example:

$P_{\geq p} [ F \; a ]$




$S^{yes} = Sat(P_{\geq 1} [ F \; a ])$

$S^{no} = Sat(\neg P_{>0} [ F \; a ])$



# Method 1 – Linear programming

- Probabilities $Pr_s^{min}(\phi_1 \; U \; \phi_2)$ for remaining states in the set $S^? = S \setminus (S^{yes} \cup S^{no})$ can be obtained as the unique solution of the following linear programming (LP) problem:

$$\text{maximize } \sum_{s \in S^?} x_s \text{ subject to the constraints :}$$

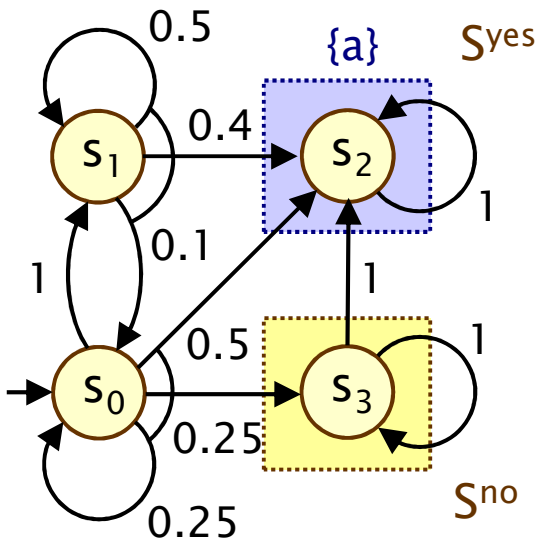
$$x_s \leq \sum_{s' \in S^?} \mu(s') \cdot x_{s'} + \sum_{s' \in S^{yes}} \mu(s')$$

$$\text{for all } s \in S^? \text{ and for all } (a, \mu) \in \delta(s)$$

- Simple case of a more general problem known as the stochastic shortest path problem [BT91]

- This can be solved with standard techniques
  - e.g. Simplex, ellipsoid method, branch-and-cut

# Example – PCTL until (LP)




Let $x_i = Pr_{s_i}^{min}(F\ a)$

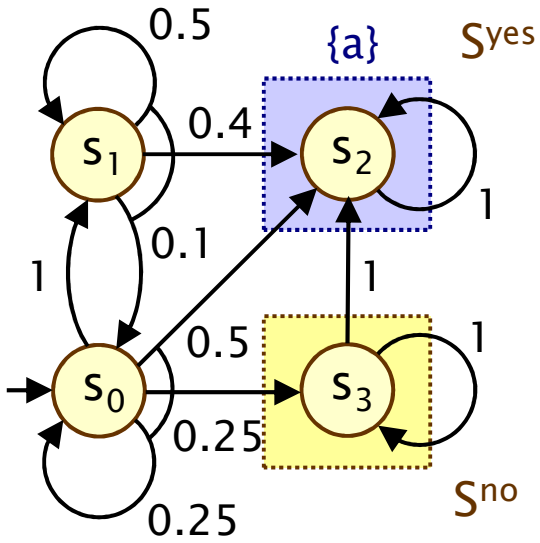
$S^{yes}$: $x_2=1$, $S^{no}$: $x_3=0$

For $S^? = \{x_0, x_1\}$ :

Maximise $x_0+x_1$ subject to constraints:

- $x_0 \leq x_1$
- $x_0 \leq 0.25 \cdot x_0 + 0.5$
- $x_1 \leq 0.1 \cdot x_0 + 0.5 \cdot x_1 + 0.4$

# Example – PCTL until (LP)



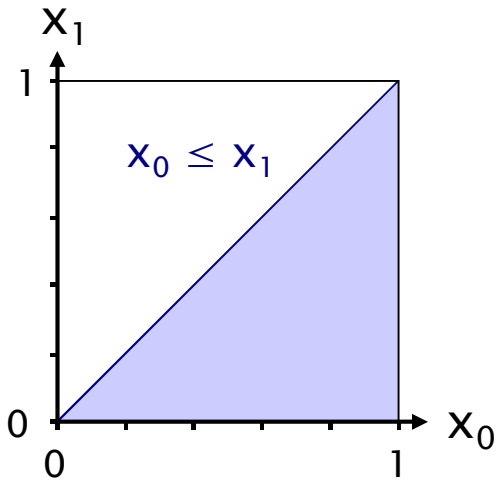

Let $x_i = Pr_{s_i}^{min}(F\ a)$

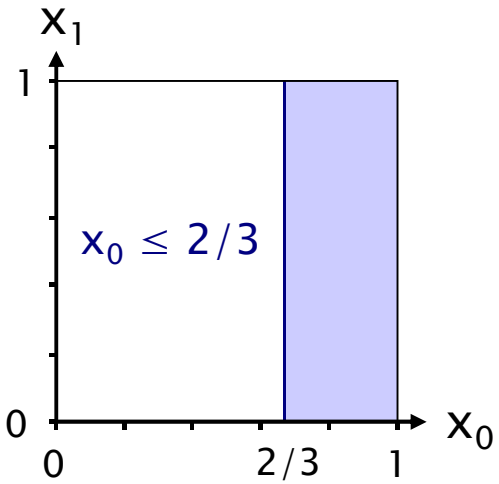
$S^{yes}$: $x_2=1$, $S^{no}$: $x_3=0$

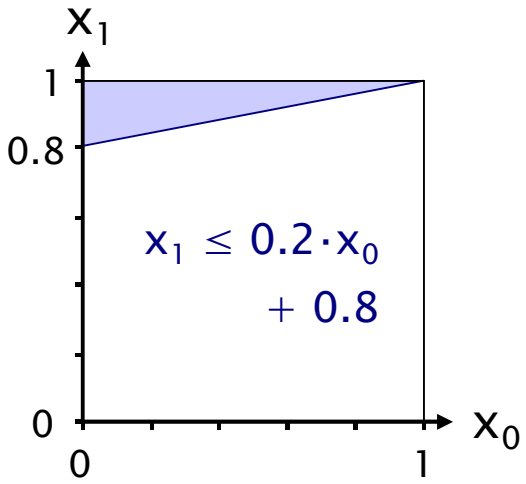
For $S^? = \{x_0, x_1\}$ :

Maximise $x_0+x_1$ subject to constraints:

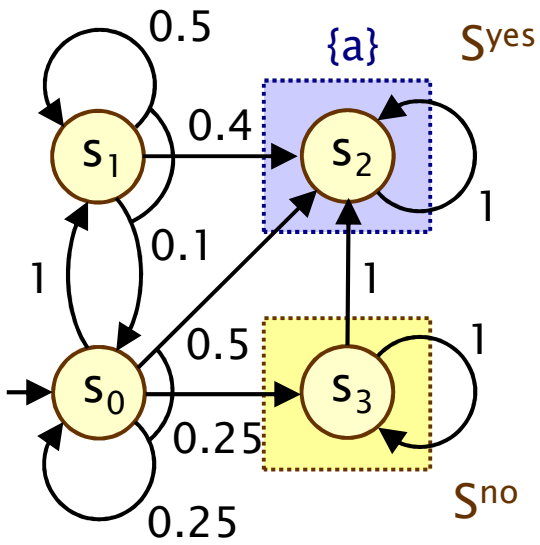
- $x_0 \leq x_1$
- $x_0 \leq 2/3$
- $x_1 \leq 0.2 \cdot x_0 + 0.8$



# Example – PCTL until (LP)



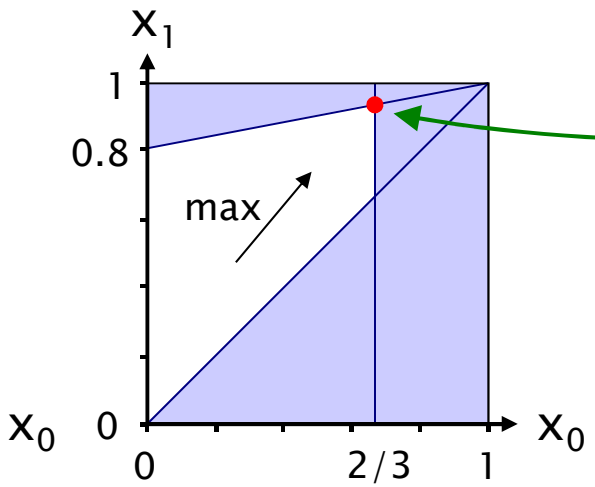

Let $x_i = Pr_{s_i}^{min}(F\ a)$

$S^{yes}$: $x_2=1$, $S^{no}$: $x_3=0$

For $S^? = \{x_0, x_1\}$ :

Maximise $x_0+x_1$ subject to constraints:

- $x_0 \leq x_1$
- $x_0 \leq 2/3$
- $x_1 \leq 0.2 \cdot x_0 + 0.8$



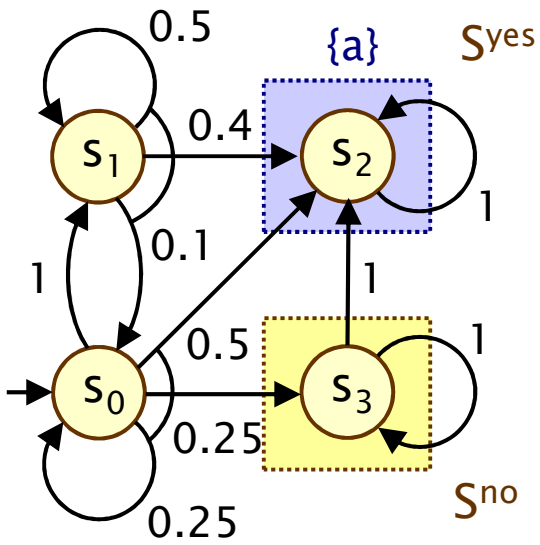

Solution:

$(x_0, x_1)$

=

$(2/3, 14/15)$

# Example – PCTL until (LP)



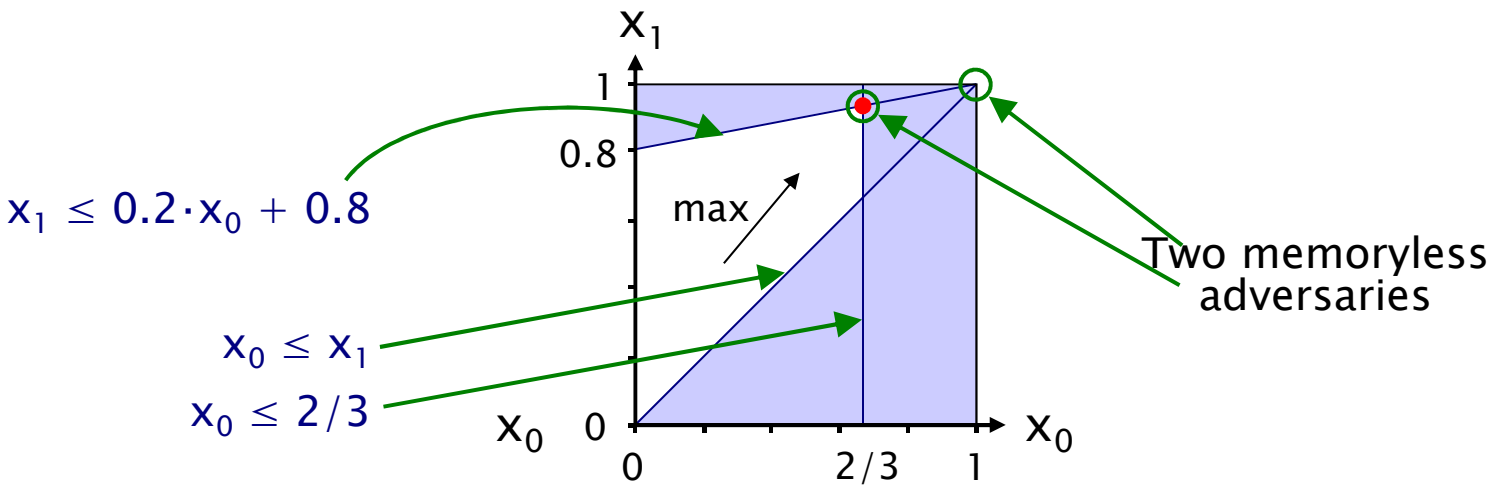

Let $x_i = Pr_{s_i}^{min}(F\ a)$

$S^{yes}$: $x_2=1$, $S^{no}$: $x_3=0$

For $S^? = \{x_0, x_1\}$ :

Maximise $x_0+x_1$ subject to constraints:

- $x_0 \leq x_1$
- $x_0 \leq 2/3$
- $x_1 \leq 0.2 \cdot x_0 + 0.8$





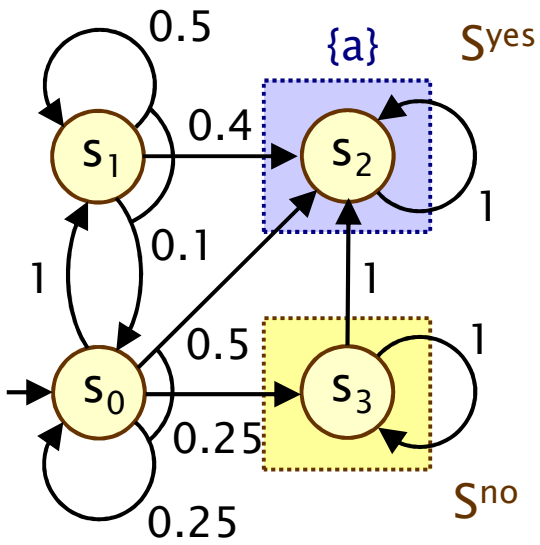
# Method 2 – Value iteration

- For probabilities $Pr_s^{min}(\phi_1 \; U \; \phi_2)$ it can be shown that:
  - $Pr_s^{min}(\phi_1 \; U \; \phi_2) = \lim_{n \to \infty} x_s^{(n)}$ where:

$$x_s^{(n)} = \begin{cases} 1 & \text{if } s \in S^{yes} \\ 0 & \text{if } s \in S^{no} \\ 0 & \text{if } s \in S^{?} \text{ and } n = 0 \\ \min_{(a,\mu) \in Steps(s)} \left( \sum_{s' \in S} \mu(s') \cdot x_{s'}^{(n-1)} \right) & \text{if } s \in S^{?} \text{ and } n > 0 \end{cases}$$

- This forms the basis for an (approximate) iterative solution
  - iterations terminated when solution converges sufficiently

# Example – PCTL until (value iteration)




Compute: $Pr_{s_i}^{min}(F\ a)$

$S^{yes} = \{x_2\}$, $S^{no} = \{x_3\}$, $S^{?} = \{x_0, x_1\}$

$[\ x_0^{(n)}, x_1^{(n)}, x_2^{(n)}, x_3^{(n)}\ ]$

n=0: [ 0, 0, 1, 0 ]

n=1: [ min(0, 0.25·0+0.5), 0.1·0+0.5·0+0.4, 1, 0 ]

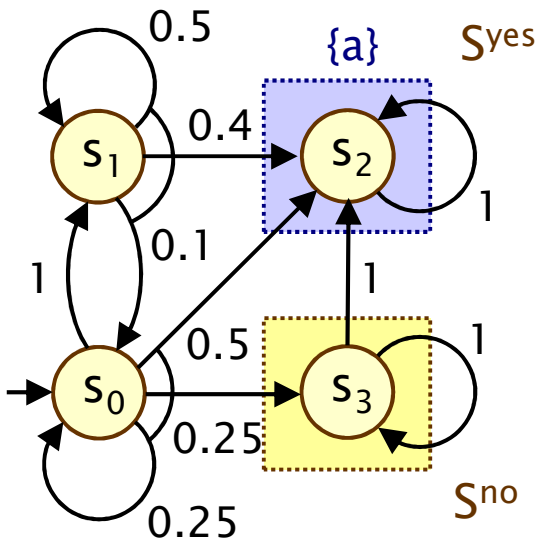
= [ 0, 0.4, 1, 0 ]

n=2: [ min(0.4, 0.25·0+0.5), 0.1·0+0.5·0.4+0.4, 1, 0 ]

= [ 0.4, 0.6, 1, 0 ]

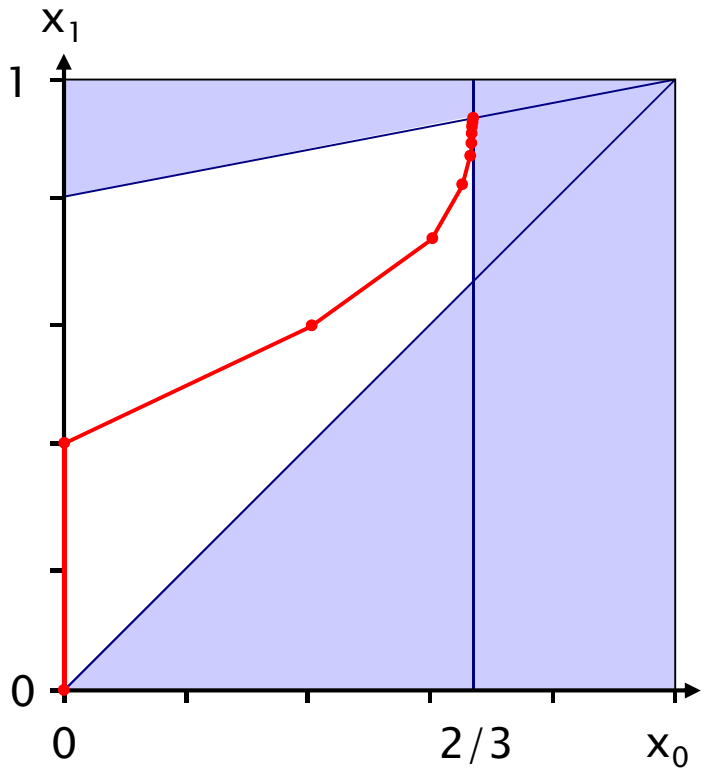
n=3: …

# Example – PCTL until (value iteration)




| | $[ x_0^{(n)}, x_1^{(n)}, x_2^{(n)}, x_3^{(n)} ]$ |
|---|---|
| n=0: | [ 0.000000, 0.000000, 1, 0 ] |
| n=1: | [ 0.000000, 0.400000, 1, 0 ] |
| n=2: | [ 0.400000, 0.600000, 1, 0 ] |
| n=3: | [ 0.600000, 0.740000, 1, 0 ] |
| n=4: | [ 0.650000, 0.830000, 1, 0 ] |
| n=5: | [ 0.662500, 0.880000, 1, 0 ] |
| n=6: | [ 0.665625, 0.906250, 1, 0 ] |
| n=7: | [ 0.666406, 0.919688, 1, 0 ] |
| n=8: | [ 0.666602, 0.926484, 1, 0 ] |
| n=9: | [ 0.666650, 0.929902, 1, 0 ] |
| | … |
| n=20: | [ 0.666667, 0.933332, 1, 0 ] |
| n=21: | [ 0.666667, 0.933332, 1, 0 ] |
| | ≈ [ 2/3, 14/15, 1, 0 ] |

# Example – Value iteration + LP




| | $[ x_0^{(n)}, x_1^{(n)}, x_2^{(n)}, x_3^{(n)} ]$ |
|---|---|
| n=0: | [ 0.000000, 0.000000, 1, 0 ] |
| n=1: | [ 0.000000, 0.400000, 1, 0 ] |
| n=2: | [ 0.400000, 0.600000, 1, 0 ] |
| n=3: | [ 0.600000, 0.740000, 1, 0 ] |
| n=4: | [ 0.650000, 0.830000, 1, 0 ] |
| n=5: | [ 0.662500, 0.880000, 1, 0 ] |
| n=6: | [ 0.665625, 0.906250, 1, 0 ] |
| n=7: | [ 0.666406, 0.919688, 1, 0 ] |
| n=8: | [ 0.666602, 0.926484, 1, 0 ] |
| n=9: | [ 0.666650, 0.929902, 1, 0 ] |
| | … |
| n=20: | [ 0.666667, 0.933332, 1, 0 ] |
| n=21: | [ 0.666667, 0.933332, 1, 0 ] |
| | ≈ [ 2/3, 14/15, 1, 0 ] |



# Method 3 – Policy iteration

- Value iteration:
  - iterates over (vectors of) probabilities
- Policy iteration:
  - iterates over strategies (“policies”)

- 1. Start with an arbitrary (memoryless) strategy σ
- 2. Compute the reachability probabilities $\underline{Pr}^{\sigma}(F\ a)$ for σ
- 3. Improve the strategy in each state
- 4. Repeat 2/3 until no change in strategy

- Termination:
  - finite number of memoryless strategies
  - improvement in (minimum) probabilities each time



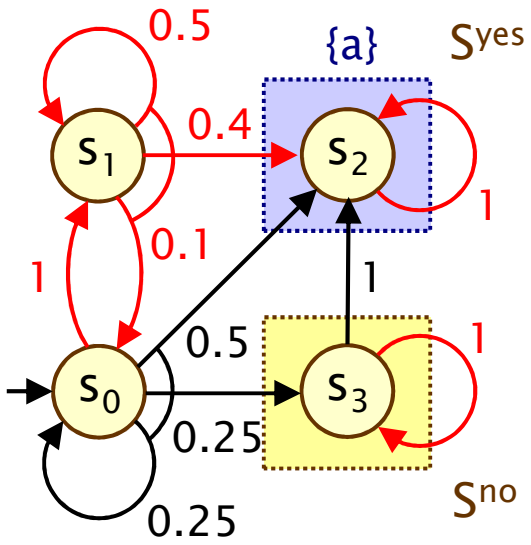
# Method 3 – Policy iteration

- 1. Start with an arbitrary (memoryless) strategy σ
  - pick an element of δ(s) for each state s ∈ S
- 2. Compute the reachability probabilities $\underline{Pr}^{\sigma}(F\ a)$ for σ
  - probabilistic reachability on a DTMC
  - i.e. solve linear equation system
- 3. Improve the strategy in each state

$$\sigma'(s) = \arg\min \left\{ \sum_{s' \in S} \mu(s') \cdot Pr^{\sigma}_{s'}(F\ a) \mid (a,\mu) \in \delta(s) \right\}$$

- 4. Repeat 2/3 until no change in strategy

# Example – Policy iteration




Arbitrary strategy σ:

Compute: $\underline{Pr}^{\sigma}(F\ a)$

Let $x_i = Pr_{s_i}^{\sigma}(F\ a)$

$x_2=1$, $x_3=0$ and:

- $x_0 = x_1$
- $x_1 = 0.1 \cdot x_0 + 0.5 \cdot x_1 + 0.4$

Solution:

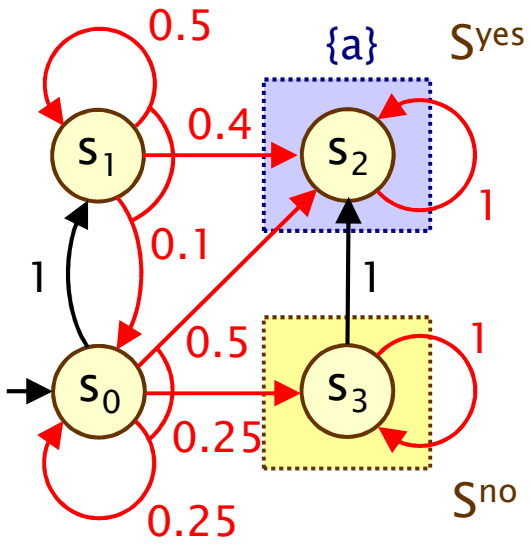
$\underline{Pr}^{\sigma}(F\ a) = [\ 1, 1, 1, 0\ ]$

Refine σ in state $s_0$:

min{1(1), 0.5(1)+0.25(0)+0.25(1)}

= min{1, 0.75} = 0.75

# Example – Policy iteration




Refined strategy σ':

Compute: $\underline{Pr}^{\sigma'}(F\ a)$

Let $x_i = Pr_{s_i}^{\sigma'}(F\ a)$

$x_2=1$, $x_3=0$ and:

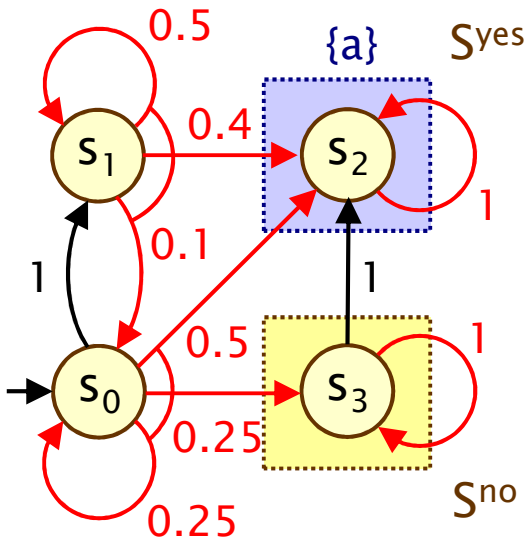
- $x_0 = 0.25 \cdot x_0 + 0.5$
- $x_1 = 0.1 \cdot x_0 + 0.5 \cdot x_1 + 0.4$

Solution:

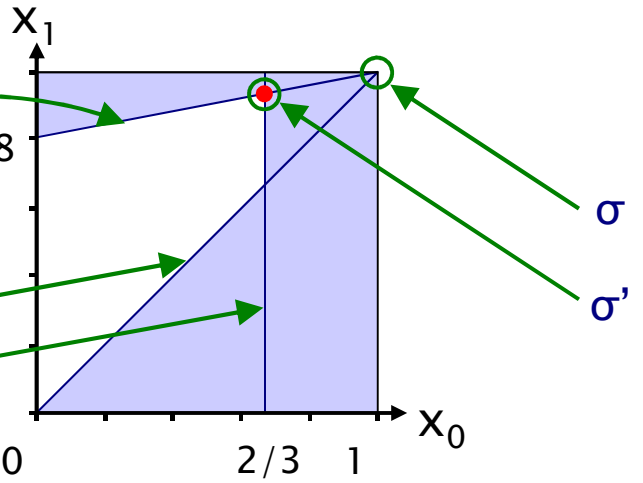
$\underline{Pr}^{\sigma'}(F\ a) = [\ 2/3,\ 14/15,\ 1,\ 0\ ]$

This is optimal

# Example – Policy iteration




$x_1 = 0.2 \cdot x_0 + 0.8$





# PCTL model checking – Summary

- Computation of set Sat(Φ) for MDP M and PCTL formula Φ
  - recursive descent of parse tree
  - combination of graph algorithms, numerical computation

- Probabilistic operator P:
  - X Φ : one matrix-vector multiplication, $O(|S|^2)$
  - $\Phi_1\ U^{\leq k}\ \Phi_2$ : k matrix-vector multiplications, $O(k|S|^2)$
  - $\Phi_1\ U\ \Phi_2$ : linear programming problem, polynomial in $|S|$ (assuming use of linear programming)

- Complexity:
  - linear in $|\Phi|$ and polynomial in $|S|$
  - S is states in MDP, assume $|\delta(s)|$ is constant



# Costs and rewards for MDPs

- We can augment MDPs with rewards (or, conversely, costs)
  - real-valued quantities assigned to states and/or transitions
  - these can have a wide range of possible interpretations
- Some examples:
  - elapsed time, power consumption, size of message queue, number of messages successfully delivered, net profit

- Extend logic PCTL with R operator, for "expected reward"
  - as for PCTL, either $R_{\sim r}$ [ … ], $R_{min=?}$ [ … ] or $R_{max=?}$ [ … ]
- Some examples:
  - $R_{min=?}$ [ $I^{=90}$ ], $R_{max=?}$ [ $C^{\leq 60}$ ], $R_{max=?}$ [ F "end" ]
  - "the minimum expected queue size after exactly 90 seconds"
  - "the maximum expected power consumption over one hour"
  - the maximum expected time for the algorithm to terminate



# Limitations of PCTL

- PCTL, although useful in practice, has limited expressivity
  - essentially: probability of reaching states in X, passing only through states in Y (and within k time-steps)
- More expressive logics can be used, for example:
  - LTL [Pnu77] – the non-probabilistic linear-time temporal logic
  - PCTL* [ASB+95,BdA95] – which subsumes both PCTL and LTL
  - both allow path operators to be combined
- In PCTL, temporal operators always appear inside $P_{\sim p}$ [...]
  - (and, in CTL, they always appear inside A or E)
  - in LTL (and PCTL*), temporal operators can be combined



# LTL + probabilities

- Same idea as PCTL: probabilities of sets of path formulae
  - for a state s of a DTMC and an LTL formula $\psi$:
  - $\text{Prob}(s, \psi) = \Pr_s \{ \omega \in \text{Path}(s) \mid \omega \models \psi \}$
  - all such path sets are measurable (see later)

- For MDPs, we can again consider lower/upper bounds
  - $p_{min}(s, \psi) = \inf_{\sigma \in \text{Adv}} \text{Prob}^{\sigma}(s, \psi)$
  - $p_{max}(s, \psi) = \sup_{\sigma \in \text{Adv}} \text{Prob}^{\sigma}(s, \psi)$
  - (for LTL formula $\psi$)

- For DTMCs or MDPs, an LTL specification often comprises an LTL (path) formula and a probability bound
  - e.g. $P_{>0.99}$ [ F ( req $\wedge$ X ack ) ]



# LTL model checking for DTMCs

- Model check LTL specification $P_{\sim p}[\ \psi\ ]$ against DTMC D

- 1. Generate a deterministic Rabin automaton (DRA) for ψ
  - build nondeterministic Büchi automaton (NBA) for ψ [VW94]
  - convert the NBA to a DRA [Saf88]
- 2. Construct product DTMC D⊗A
- 3. Identify accepting BSCCs of D⊗A
- 4. Compute probability of reaching accepting BSCCs
  - from all states of the D⊗A
- 5. Compare probability for $(s, q_s)$ against p for each s

- Qualitative LTL model checking – no probabilities needed



# PCTL* model checking

- PCTL* syntax:
  - $\phi$ ::= true | a | $\phi \wedge \phi$ | $\neg\phi$ | $P_{\sim p}$ [ $\psi$ ]
  - $\psi$ ::= $\phi$ | $\psi \wedge \psi$ | $\neg\psi$ | X $\psi$ | $\psi$ U $\psi$
- Example:
  - $P_{>p}$ [ GF ( send → $P_{>0}$ [ F ack ] ) ]

- PCTL* model checking algorithm
  - bottom-up traversal of parse tree for formula (like PCTL)
  - to model check $P_{\sim p}$ [ $\psi$ ]:
    - replace maximal state subformulae with atomic propositions
    - (state subformulae already model checked recursively)
    - modified formula $\psi$ is now an LTL formula
    - which can be model checked as for LTL



# LTL model checking for MDPs

- Model check LTL specification $P_{\sim p}[\ \psi\ ]$ against MDP M

- 1. Convert problem to one needing maximum probabilities
  - e.g. convert $P_{>p}[\ \psi\ ]$ to $P_{<1-p}[\ \neg\psi\ ]$
- 2. Generate a DRA for $\psi$ (or $\neg\psi$)
  - build nondeterministic Büchi automaton (NBA) for $\psi$ [VW94]
  - convert the NBA to a DRA [Saf88]
- 3. Construct product MDP $M \otimes A$
- 4. Identify accepting end components (ECs) of $M \otimes A$
- 5. Compute max. probability of reaching accepting ECs
  - from all states of the $D \otimes A$
- 6. Compare probability for $(s, q_s)$ against p for each s



# Complexity

- Complexity of model checking LTL formula ψ on DTMC D
  - is doubly exponential in |ψ| and polynomial in |D|
- Converting LTL formula ψ to DRA A
  - for some LTL formulae of size n, size of smallest DRA is $2^{2^n}$
- In total: O(poly(|D|,|A|))
- In practice: |ψ| is small and |D| is large
- Can be reduced to single exponential in |ψ|
  - see e.g. [CY88,CY95]

- Complexity of model checking LTL formula ψ on MDP M
  - is doubly exponential in |ψ| and polynomial in |M|
  - unlike DTMCs, this cannot be improved upon



# Overview (Part 2)

- Introduction
- Model checking for Markov decision processes (MDPs)
  - MDPs: definition
  - Paths, strategies & probability spaces
  - PCTL model checking
- Strategy synthesis for MDPs
  - Properties and objectives
  - Verification vs synthesis
  - Case study: Dynamic power management
- Summary and further work



# Verification vs synthesis

- Majority of research to date has focused on verification
  - scalability and performance of algorithms
  - extending expressiveness of models and logics
- Some work to date on counterexamples
  - but difficult to represent them compactly

- In this lecture, focus on simpler problem of strategy synthesis
  - can we find a strategy to guarantee that a given quantitative property is satisfied?
  - advantage: correct-by-construction
- Not a well known fact…
  - incidentally, can reuse the verification algorithms…

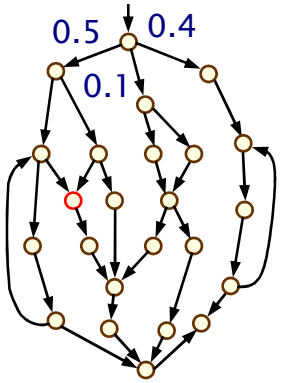
# Quantitative (probabilistic) verification

Automatic verification and strategy synthesis from quantitative properties for probabilistic models

System



Probabilistic model
e.g. Markov chain



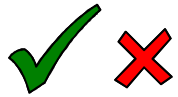

System require-ments

$P_{<0.01}$ [ $F^{\leq t}$ fail]

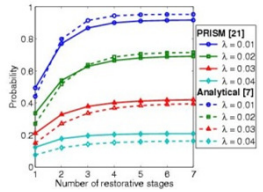
Probabilistic temporal logic specification
e.g. PCTL, CSL, LTL

Probabilistic model checker
e.g. PRISM

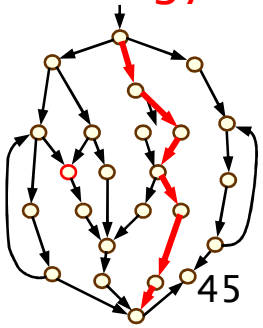
Result



Quantitative results

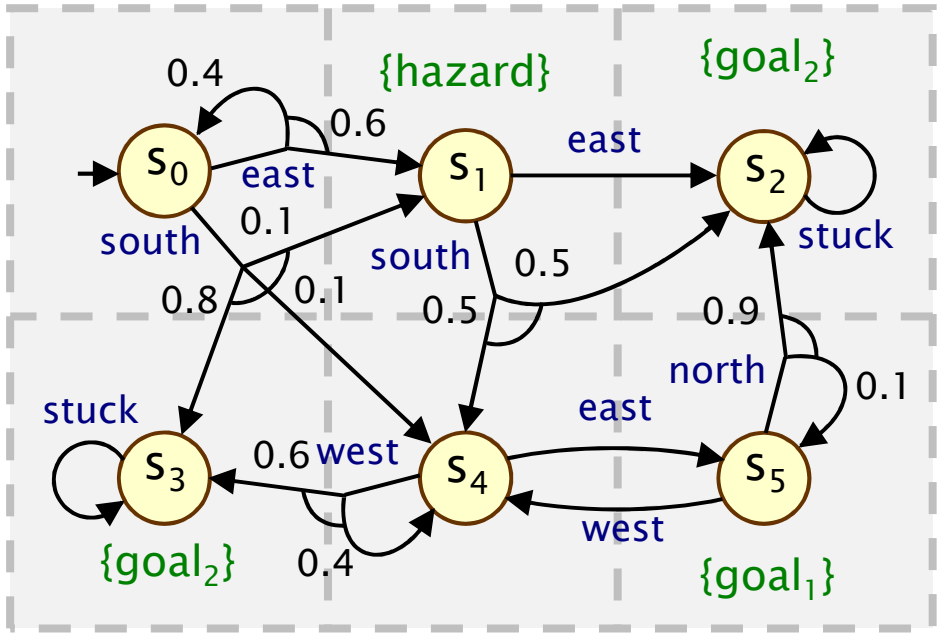



Strategy



# Running example

- Example MDP
  - robot moving through terrain divided into 3 x 2 grid




States:

$s_0$, $s_1$, $s_2$, $s_3$, $s_4$, $s_5$

Actions:

north, east, south, west, stuck

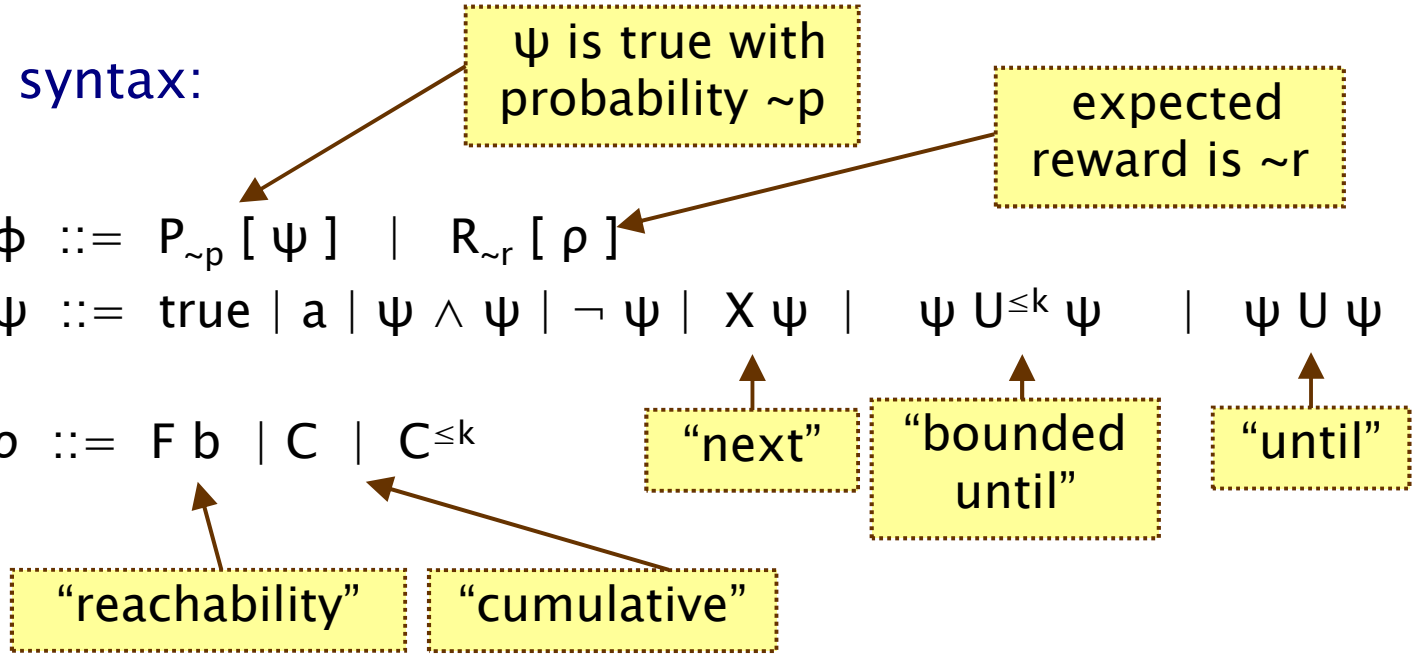
Labels

(atomic propositions):

hazard, $goal_1$, $goal_2$

# Properties and objectives

- The syntax:
  - $\phi$ ::= $P_{\sim p}[\psi]$ | $R_{\sim r}[\rho]$
  - $\psi$ ::= true | a | $\psi \wedge \psi$ | $\neg \psi$ | $X\,\psi$ | $\psi\, U^{\leq k}\, \psi$ | $\psi\, U\, \psi$
  - $\rho$ ::= $F\,b$ | $C$ | $C^{\leq k}$




  - where b is an atomic proposition, used to identify states of interest, $p \in [0,1]$ is a probability, $\sim \in \{<,>,\leq,\geq\}$, $k \in \mathbb{N}$, and $r \in \mathbb{R}_{\geq 0}$
  - $F\,b \equiv \text{true}\ U\ b$
- We refer to $\phi$ as property, $\psi$ and $\rho$ as objectives
  - (branching time more challenging for synthesis)



# Properties and objectives

- Semantics of the probabilistic operator P
  - can only define probabilities for a specific strategy σ
  - $s \models P_{\sim p}[\psi]$ means "the probability, from state s, that ψ is true for an outgoing path satisfies ~p for all strategies σ"
  - formally $s \models P_{\sim p}[\psi] \Leftrightarrow Pr_s^{\sigma}(\psi) \sim p$ for all strategies σ
  - where we use $Pr_s^{\sigma}(\psi)$ to denote $Pr_s^{\sigma}\{\omega \in Path_s^{\sigma} \mid \omega \models \psi\}$
- $R_{\sim r}[\cdot]$ means "the expected value of · satisfies ~r"
- Some examples:
  - $P_{\geq 0.4}[F \text{ "goal"}]$ "probability of reaching goal is at least 0.4"
  - $R_{<5}[C^{\leq 60}]$ "expected power consumption over one hour is below 5"
  - $R_{\leq 10}[F \text{ "end"}]$ "expected time to termination is at most 10"



# Verification and strategy synthesis

- The verification problem is:
  - Given an MDP M and a property $\phi$, does M satisfy $\phi$ under any possible strategy $\sigma$?
- The synthesis problem is dual:
  - Given an MDP M and a property $\phi$, find, if it exists, a strategy $\sigma$ such that M satisfies $\phi$ under $\sigma$
- Verification and strategy synthesis is achieved using the same techniques, namely computing optimal values for probability objectives:
  - $Pr_s^{min}(\psi) = \inf_\sigma Pr_s^\sigma(\psi)$
  - $Pr_s^{max}(\psi) = \sup_\sigma Pr_s^\sigma(\psi)$
  - and similarly for expectations



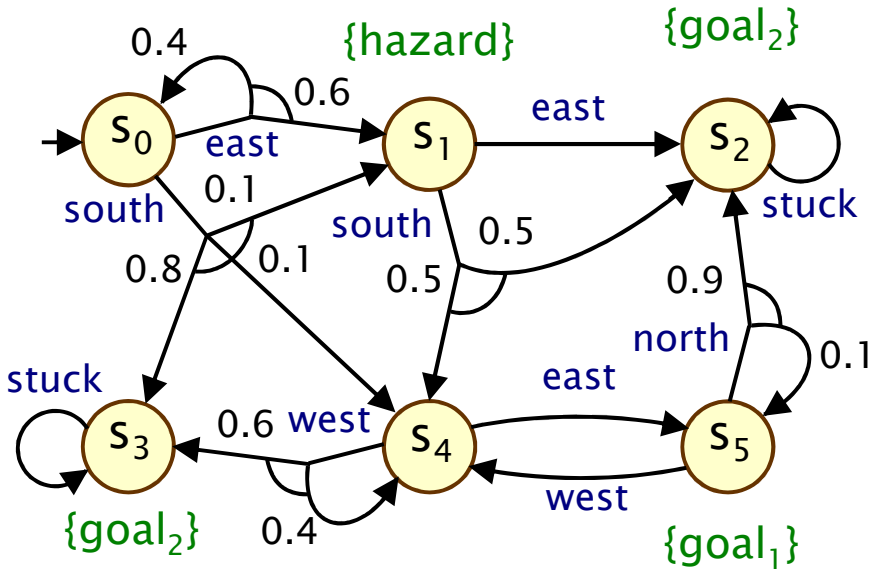
# Computing reachability for MDPs

- Computation of probabilities $Pr_s^{max}(F\ b)$ for all $s \in S$

- Step 1: pre-compute all states where probability is 1 or 0
  - graph-based algorithms, yielding sets $S^{yes}$, $S^{no}$
- Step 2: compute probabilities for remaining states ($S^?$)
  - (i) solve linear programming problem
  - (i) approximate with value iteration
  - (iii) solve with policy (strategy) iteration

- 1. Precomputation:
  - algorithm Prob1E computes $S^{yes}$
    - there exists a strategy for which the probability of "F b" is 1
  - algorithm Prob0A computes $S^{no}$
    - for all strategies, the probability of satisfying "F b" is 0

# Example – Reachability



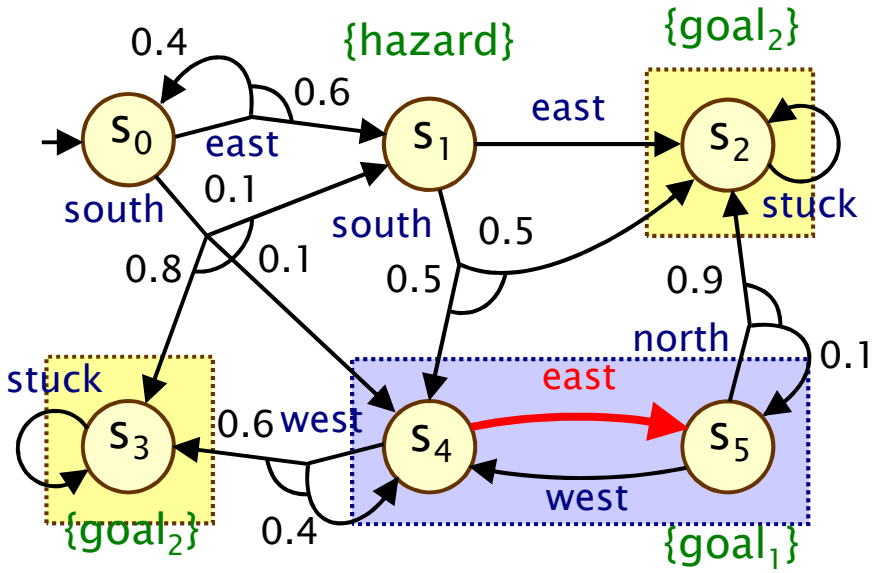

Example:

$P_{\geq 0.4}$ [ F $goal_1$ ]

So compute:

$Pr_s^{max}$(F $goal_1$)

# Example – Precomputation




Syes

Sno

Example:

$P_{\geq 0.4}$ [ F $goal_1$ ]

So compute:

$Pr_s^{max}$(F $goal_1$)



# Reachability for MDPs

- 2. Numerical computation
  - compute probabilities $Pr_s^{max}(F\ b)$
  - for remaining states in $S^? = S \setminus (S^{yes} \cup S^{no})$
  - obtained as the unique solution of the linear programming (LP) problem:

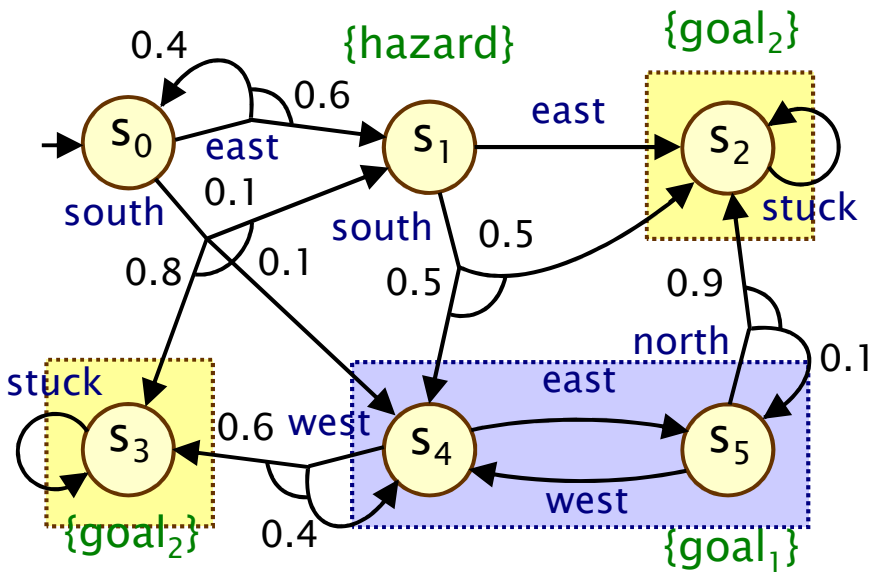
minimize $\sum_{s \in S^?} x_s$ subject to the constraints:

$$x_s \geq \sum_{s' \in S^?} \delta(s,a)(s') \cdot x_{s'} + \sum_{s' \in S^{yes}} \delta(s,a)(s')$$

for all $s \in S^?$ and for all $a \in A(s)$

- This can be solved with standard techniques
  - e.g. Simplex, ellipsoid method, branch-and-cut

# Example – Reachability (LP)




Example:

$P_{\geq 0.4}$ [ F $goal_1$ ]

So compute:

$Pr_s^{max}$(F $goal_1$)

Let $x_i = Pr_{s_i}^{max}$(F $goal_1$)

$S^{yes}$: $x_4=x_5=1$

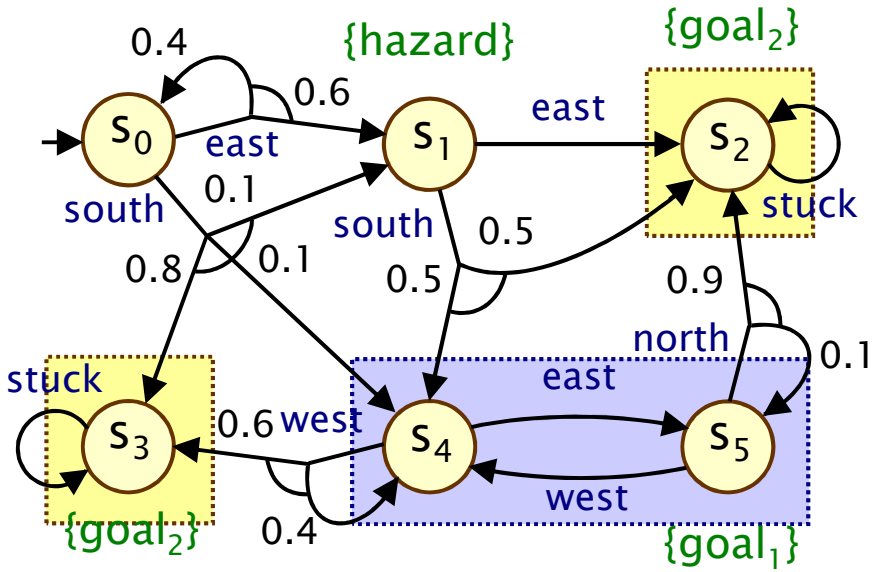
$S^{no}$: $x_2=x_3=0$

For $S^? = \{x_0, x_1\}$ :

Minimise $x_0+x_1$ subject to:

- $x_0 \geq 0.4 \cdot x_0 + 0.6 \cdot x_1$ (east)
- $x_0 \geq 0.1 \cdot x_1 + 0.1$ (south)
- $x_1 \geq 0.5$ (south)
- $x_1 \geq 0$ (east)

# Example – Reachability (LP)




Let $x_i = Pr_{s_i}^{max}(F\ goal_1)$

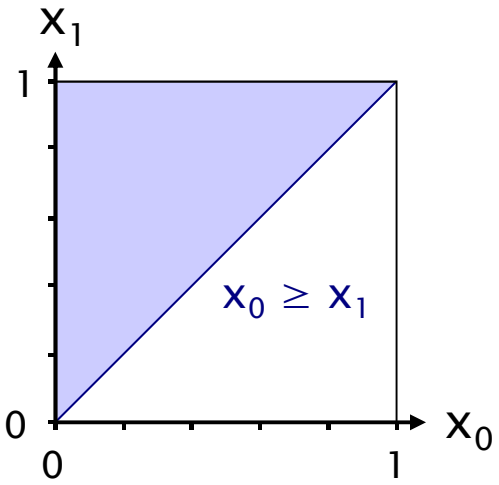
$S^{yes}$: $x_4 = x_5 = 1$

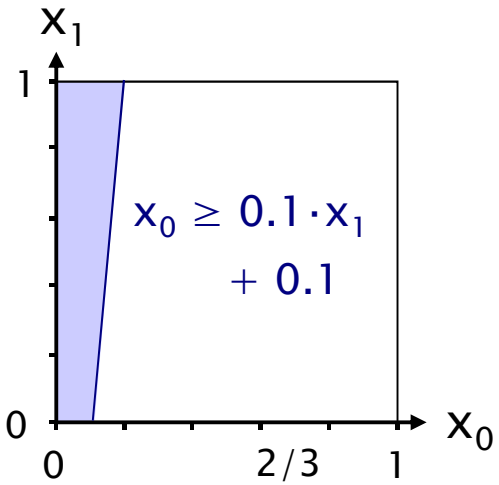
$S^{no}$: $x_2 = x_3 = 0$

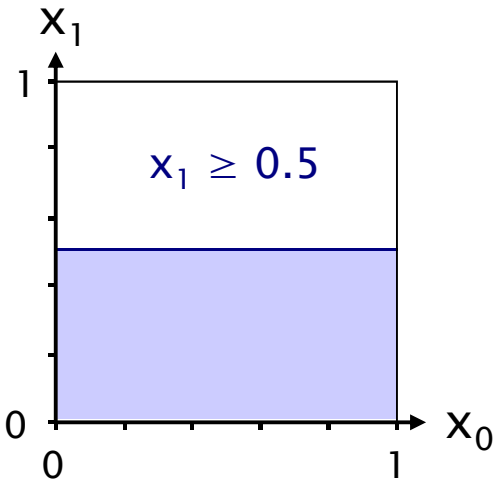
For $S^? = \{x_0, x_1\}$ :

Minimise $x_0 + x_1$ subject to:

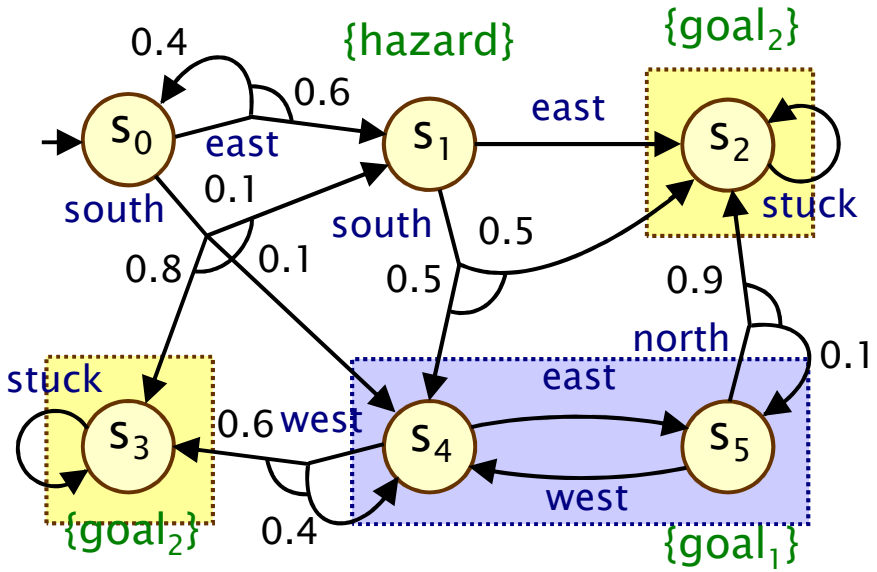
- $x_0 \geq x_1$ (east)
- $x_0 \geq 0.1 \cdot x_1 + 0.1$ (south)
- $x_1 \geq 0.5$ (south)



# Example – Reachability (LP)




Let $x_i = Pr_{s_i}^{max}(F\ goal_1)$

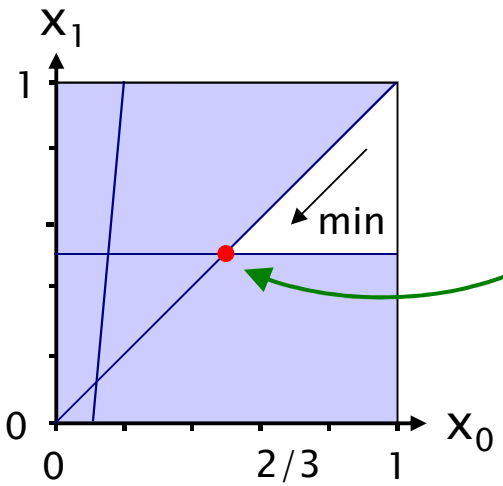
$S^{yes}$: $x_4 = x_5 = 1$

$S^{no}$: $x_2 = x_3 = 0$

For $S^? = \{x_0, x_1\}$ :

Minimise $x_0 + x_1$ subject to:

- $x_0 \geq x_1$
- $x_0 \geq 0.1 \cdot x_1 + 0.1$
- $x_1 \geq 0.5$




Solution:

$(x_0, x_1) = (0.5, 0.5)$

i.e.

$Pr_{s_0}^{max}(F\ goal_1) = 0.5$

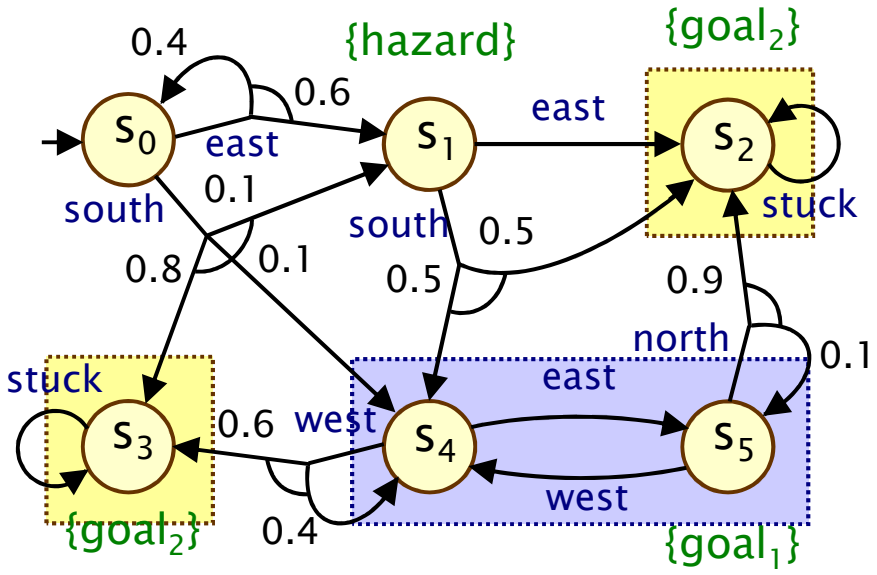
# Reachability for MDPs

- 2. Numerical computation (alternative method)
  - value iteration
  - it can be shown that: $Pr_s^{max}(F\ b) = \lim_{n\to\infty} x_s^{(n)}$ where:

$$x_s^{(n)} = \begin{cases} 1 & \text{if } s \in S^{yes} \\ 0 & \text{if } s \in S^{no} \\ 0 & \text{if } s \in S^{?} \text{ and } n = 0 \\ \max\left\{ \sum_{s' \in S} \delta(s,a)(s') \cdot x_{s'}^{(n-1)} \mid a \in A(s) \right\} & \text{if } s \in S^{?} \text{ and } n > 0 \end{cases}$$

- Approximate iterative solution technique
  - iterations terminated when solution converges sufficiently

# Example – Reachability (val. iter.)




Compute: $Pr_s^{max}(F\ goal_1)$

$S^{yes}$: $x_4=x_5=1$

$S^{no}$: $x_2=x_3=0$

$S^{?} = \{x_0, x_1\}$

$[\ x_0^{(n)}, x_1^{(n)}, x_2^{(n)}, x_3^{(n)}, x_4^{(n)}, x_5^{(n)}\ ]$

n=0: $[\ 0, 0, 0, 0, 1, 1\ ]$

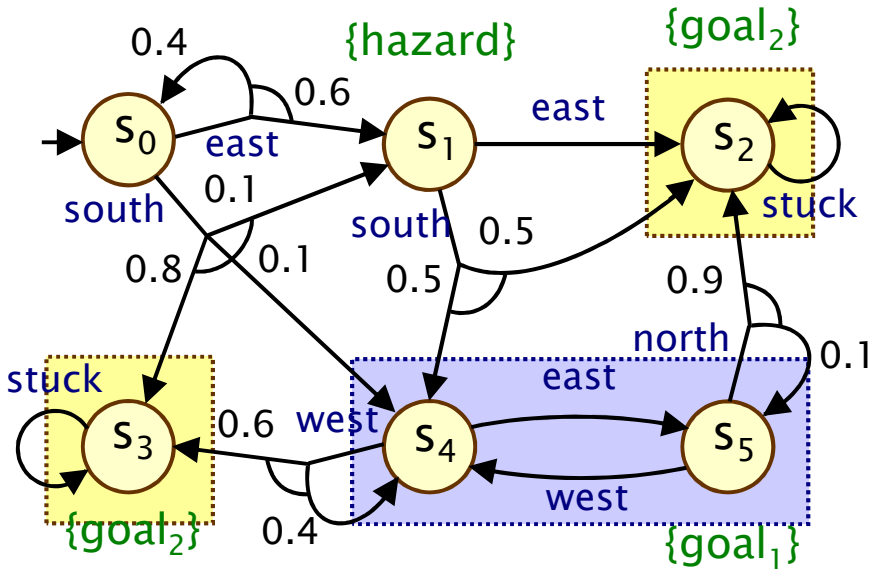
n=1: $[\ \max(0.6\cdot 0+0.4\cdot 0,\ 0.1\cdot 0+0.1\cdot 1+0.8\cdot 0),\ \max(0, 0.5),\ 0, 0, 1, 1\ ]$

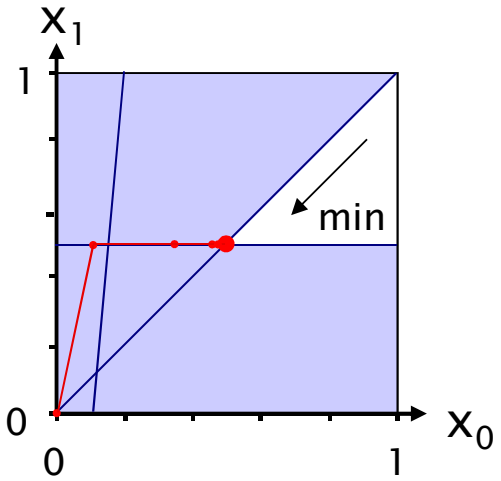
$= [\ 0.1, 0.5, 0, 0, 1, 1\ ]$

n=2: $[\ \max(0.6\cdot 0.5+0.4\cdot 0.1,\ 0.1\cdot 0.5+0.1\cdot 1+0.8\cdot 0),\ \max(0, 0.5),\ 0, 0, 1, 1\ ]$

$= [\ 0.34, 0.5, 0, 0, 1, 1\ ]$

# Example – Reachability (val. iter.)




$[x_0^{(n)}, x_1^{(n)}, x_2^{(n)}, x_3^{(n)}, x_4^{(n)}, x_5^{(n)}]$

n=0: [0, 0, 0, 0, 1, 1]

n=1: [0.1, 0.5, 0, 0, 1, 1]

n=2: [0.34, 0.5, 0, 0, 1, 1]

n=3: [0.436, 0.5, 0, 0, 1, 1]

n=4: [0.4744, 0.5, 0, 0, 1, 1]

n=5: [0.48976, 0.5, 0, 0, 1, 1]

n=6: [0.495904, 0.5, 0, 0, 1, 1]

n=7: [0.4983616, 0.5, 0, 0, 1, 1]

n=8: [0.49934464, 0.5, 0, 0, 1, 1]

…

n=16: [0.49999957, 0.5, 0, 0, 1, 1]

n=17: [0.49999982, 0.5, 0, 0, 1, 1]

… ≈ [0.5 0.5, 0, 0, 1, 1]



# Memoryless strategies

- Memoryless strategies suffice for probabilistic reachability
  - i.e. there exist memoryless strategies $\sigma_{min}$ & $\sigma_{max}$ such that:
  - $Prob^{\sigma_{min}}(s, F\ a) = p_{min}(s, F\ a)$ for all states $s \in S$
  - $Prob^{\sigma_{max}}(s, F\ a) = p_{max}(s, F\ a)$ for all states $s \in S$

- Construct strategies from optimal solution:

$$\sigma_{min}(s) = \text{argmin} \left\{ \sum_{s' \in S} \mu(s') \cdot p_{min}(s', F\ a) \mid (a, \mu) \in \textbf{Steps}(s) \right\}$$

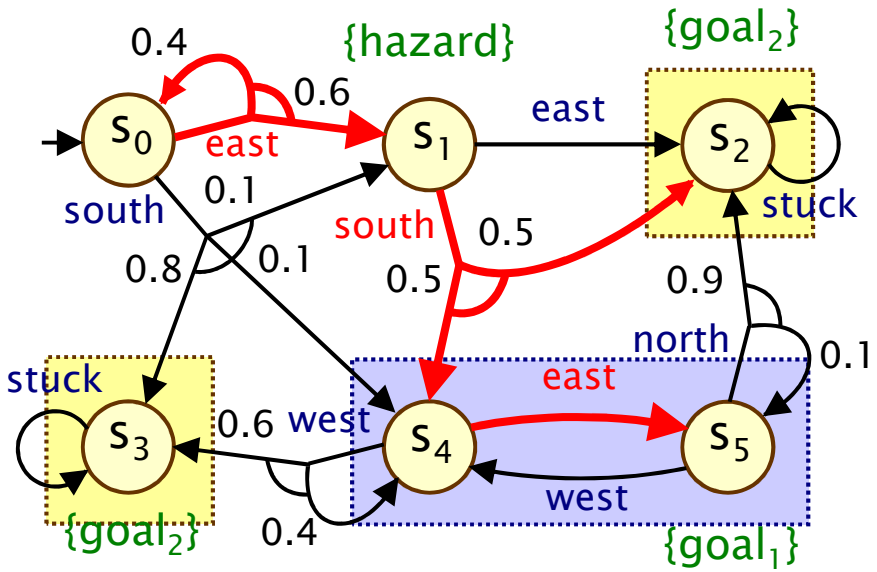
$$\sigma_{max}(s) = \text{argmax} \left\{ \sum_{s' \in S} \mu(s') \cdot p_{max}(s', F\ a) \mid (a, \mu) \in \textbf{Steps}(s) \right\}$$



# Strategy synthesis

- Compute optimal probabilities $Pr_s^{max}(F\ b)$ for all $s \in S$
- To compute the optimal strategy $\sigma^*$, choose the locally optimal action in each state
  - in general depends on the method used to compute the optimal probabilities
- For reachability
  - memoryless strategies suffice
- For step-bounded reachability
  - need finite-memory strategies
  - typically requires backward computation for a fixed number of steps

# Example – Strategy




Optimal strategy:

$s_0$ : east

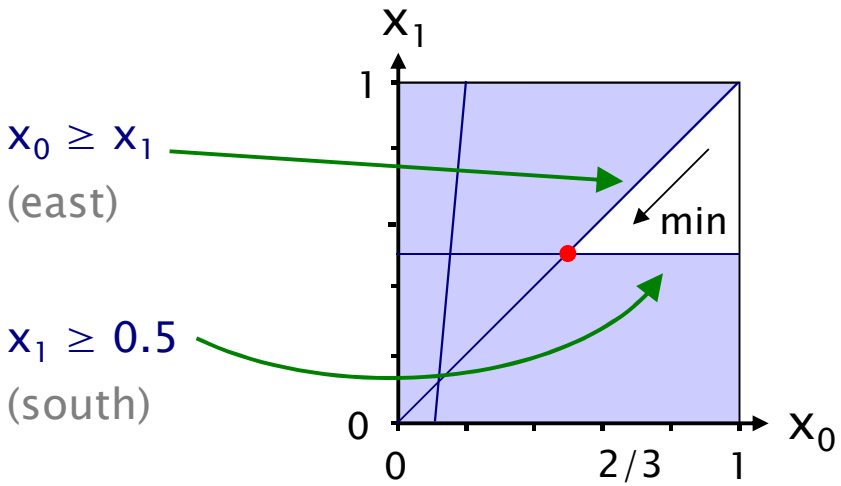
$s_1$ : south

$s_2$ : –

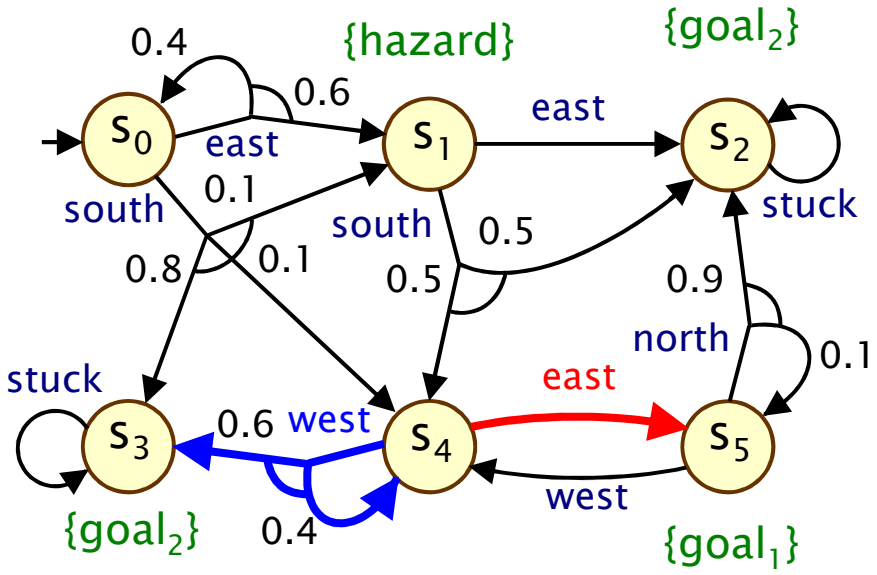
$s_3$ : –

$s_4$ : east

$s_5$ : –



# Example – Bounded reachability




Example:

$P_{max=?}$ [ $F^{\leq 3}$ $goal_2$ ]

So compute:

$Pr_s^{max}(F^{\leq 3}\ goal_2) = 0.99$

Optimal strategy is finite-memory:

$s_4$ (after 1 step): east

$s_4$ (after 2 steps): west



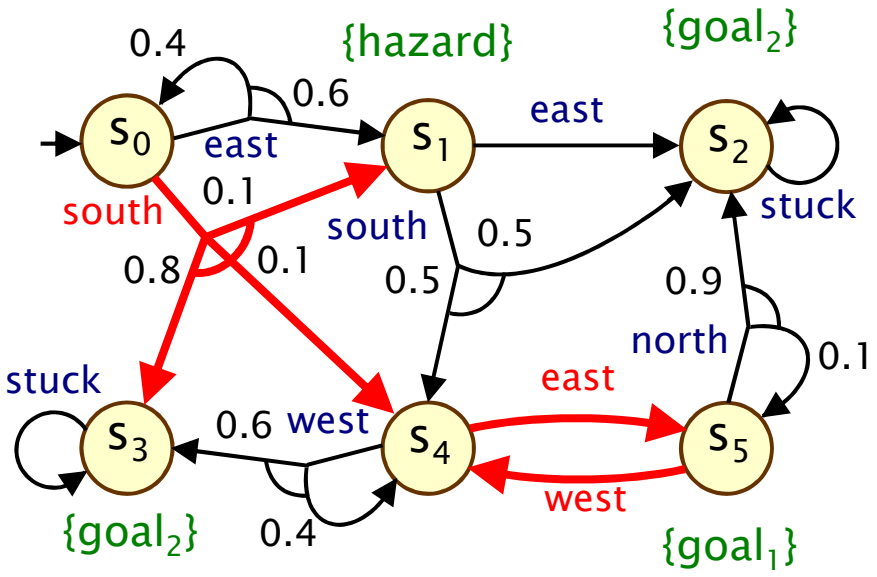
# Strategy synthesis for LTL objectives

- Reduce to the problem of reachability on the product of MDP M and an omega-automaton representing $\psi$
  - for example, deterministic Rabin automaton (DRA)
- Need only consider computation of maximum probabilities $Pr_s^{max}(\psi)$
  - since $Pr_s^{min}(\psi) = 1 - Pr_s^{max}(\neg\psi)$
- To compute the optimal strategy $\sigma^*$
  - find memoryless deterministic strategy on the product
  - convert to finite-memory strategy with one mode for each state of the DRA for $\psi$

# Example – LTL

- $P_{\geq 0.05}$ [ (G ¬hazard) ∧ (GF $goal_1$) ]
  - avoid hazard and visit $goal_1$ infinitely often

- $Pr_{s_0}^{max}$((G ¬hazard) ∧ (GF $goal_1$)) = 0.1




Optimal strategy:
(in this instance, memoryless)

$s_0$ : south

$s_1$ : –

$s_2$ : –

$s_3$ : –

$s_4$ : east

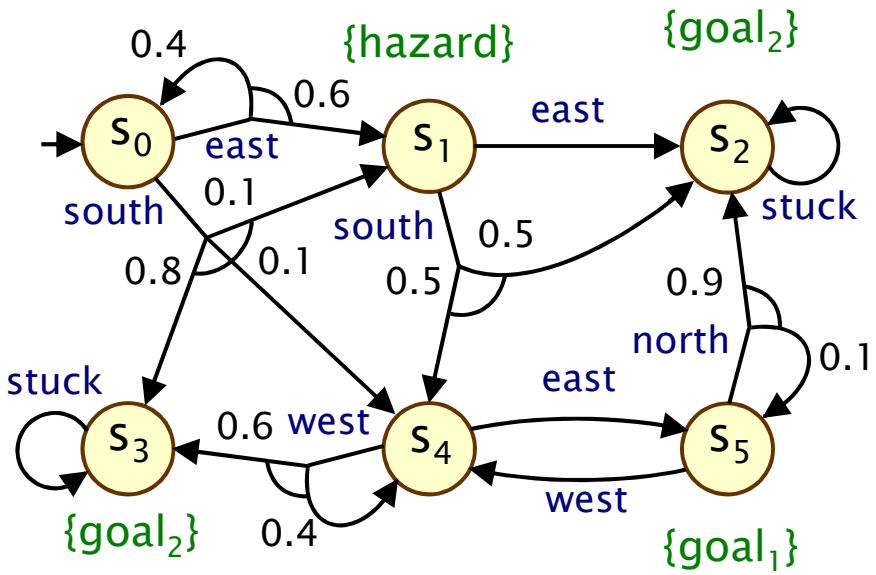
$s_5$ : west

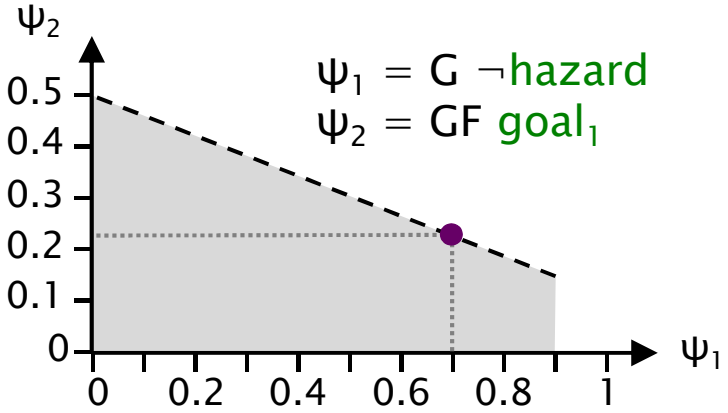


# Multi-objective strategy synthesis

- Consider conjunctions of probabilistic LTL formulas $P_{\sim p}$ [ψ]
  - require all conjuncts to be satisfied
- Reduce to a multi-objective reachability problem on the product of MDP M and the omega-automata representing the conjuncts
  - convert (by negation) to formulas with upper probability bounds (≥, >), then to DRA
  - need to consider all combinations of objectives
- The problem can be solved using LP methods [TACAS07] or via approximations to Pareto curve [ATVA12]
  - strategies may be finite memory and randomised
- Continue as for single-objectives to compute the strategy σ*
  - find memoryless deterministic strategy on the product
  - convert to finite-memory strategy

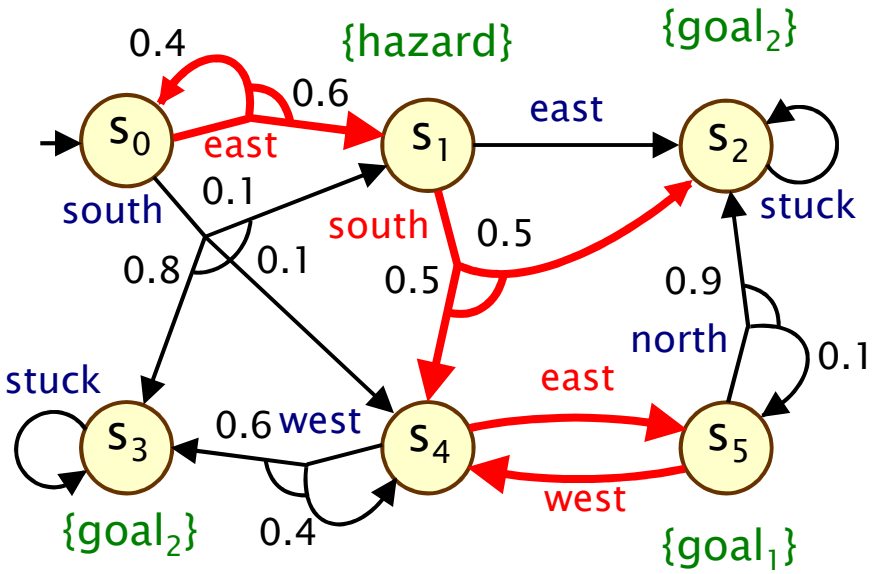
# Example – Multi-objective

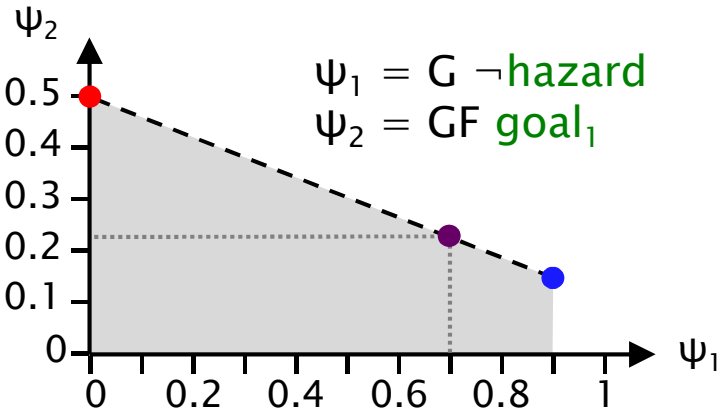



- Multi-objective formula
  - $P_{\geq 0.7}$ [ G ¬hazard ] ∧ $P_{\geq 0.2}$ [ GF $goal_1$ ] ? True (achievable)
- Numerical query
  - $P_{max=?}$ [ GF $goal_1$ ] such that $P_{\geq 0.7}$ [ G ¬hazard ] ? ~0.2278
- Pareto query
  - for $P_{max=?}$ [ G ¬hazard ] ∧ $P_{max=?}$ [ GF $goal_1$ ] ?

# Example – Multi-objective strategies




Strategy 1
(deterministic)

$s_0$ : east

$s_1$ : south

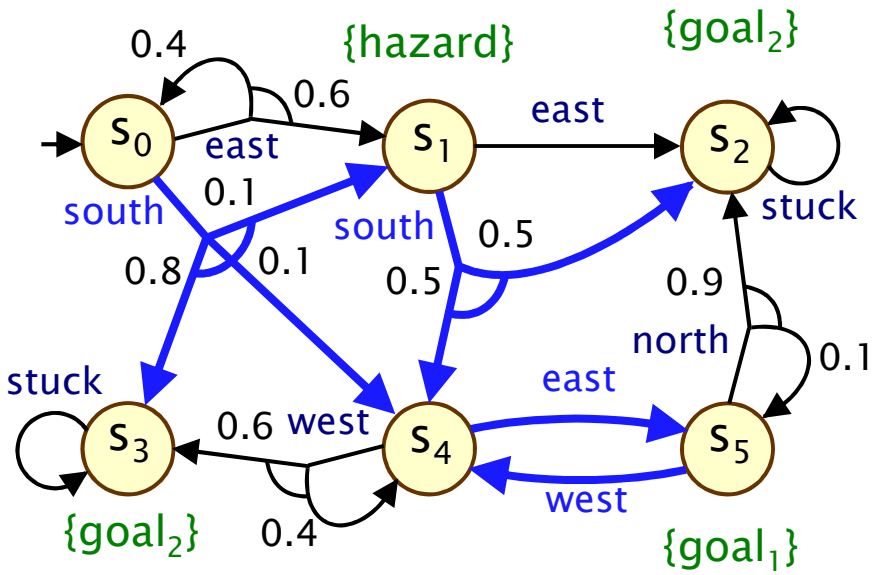
$s_2$ : –

$s_3$ : –

$s_4$ : east

$s_5$ : west

# Example – Multi-objective strategies




Strategy 2
(deterministic)

$s_0$ : south

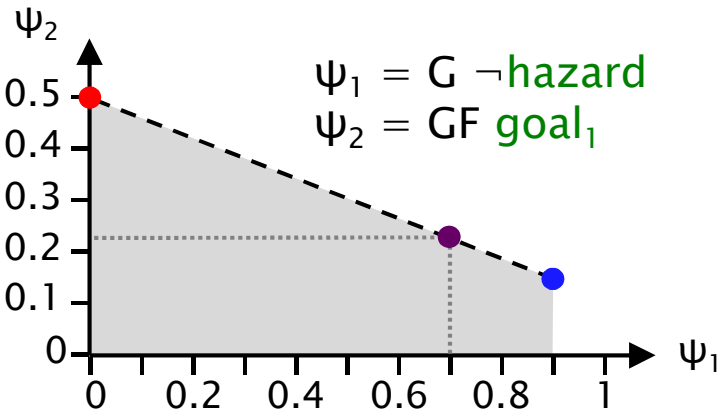
$s_1$ : south

$s_2$ : –

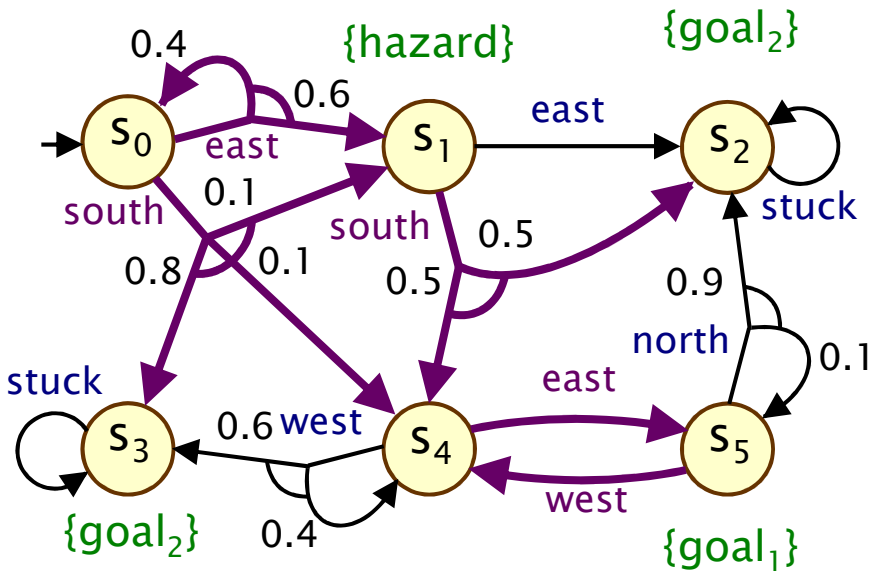
$s_3$ : –

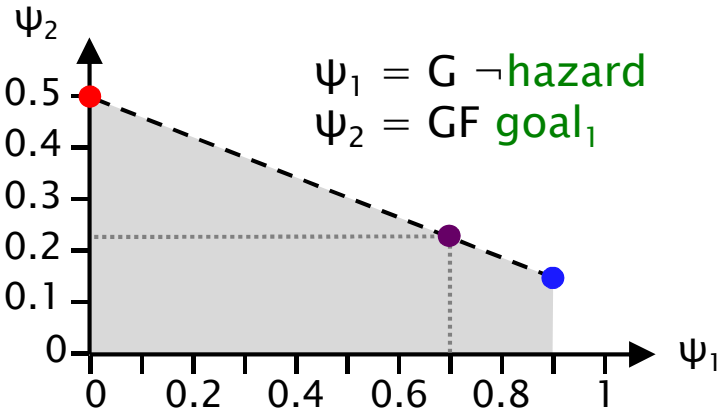
$s_4$ : east

$s_5$ : west



# Example – Multi-objective strategies




Optimal strategy:

(randomised)

$s_0$ : 0.3226 : east

0.6774 : south

$s_1$ : 1.0 : south

$s_2$ : –

$s_3$ : –

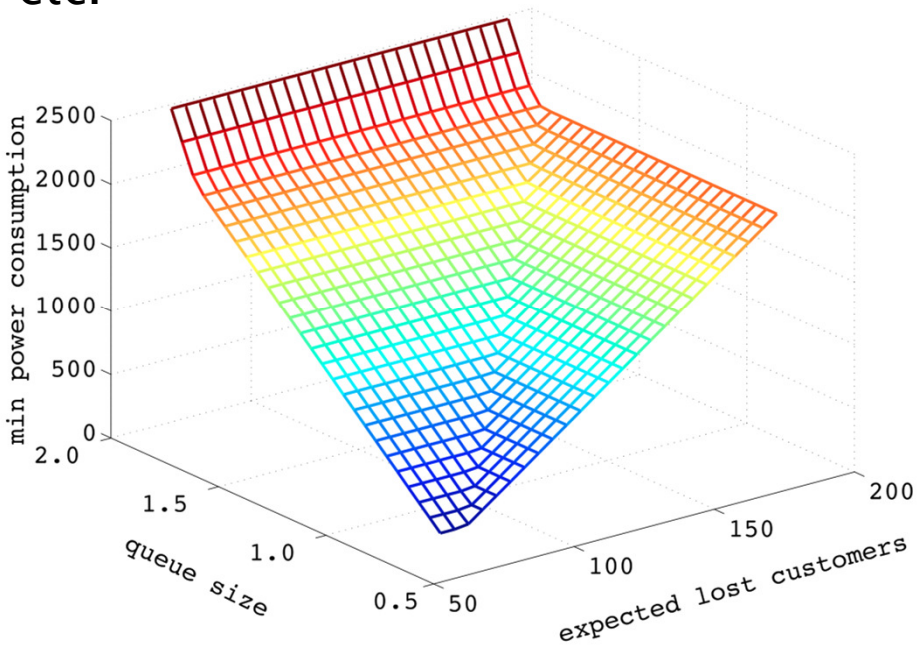
$s_4$ : 1.0 : east

$s_5$ : 1.0 : west

# Case study: Dynamic power management

- Synthesis of dynamic power management schemes
  - for an IBM TravelStar VP disk drive
  - 5 different power modes: active, idle, idlelp, stby, sleep
  - power manager controller bases decisions on current power mode, disk request queue, etc.

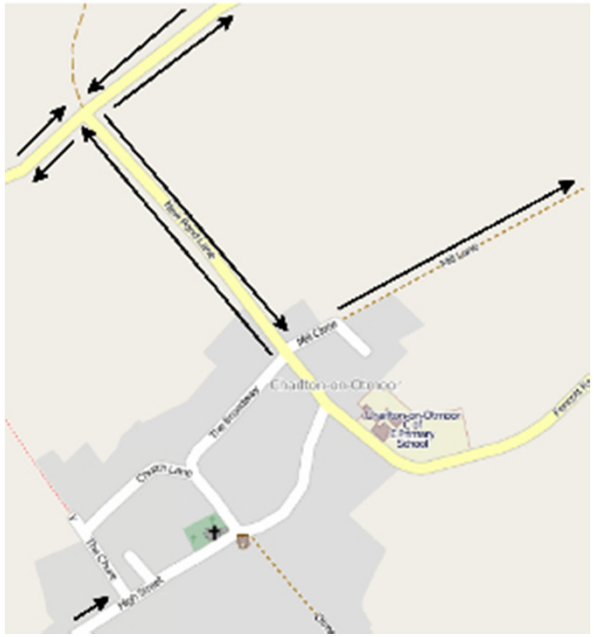
- Build controllers that
  - minimise energy consumption, subject to constraints on e.g.
  - probability that a request waits more than K steps
  - expected number of lost disk requests




- See: lab and http://www.prismmodelchecker.org/files/tacas11/

# Case study: Autonomous urban driving

- Inspired by DARPA challenge
  - represent map data as a stochastic game, with environment able to select hazards
  - express goals as conjunctions of probabilistic and reward properties
  - e.g. “maximise probability of avoiding hazards and minimise time to reach destination”
- Solution (PRISM-games)
  - synthesise a probabilistic strategy to achieve the multi-objective goal
  - enable the exploration of trade-offs between subgoals
- Applied to synthesise driving strategies for English villages
  - new project starting, joint with Paul Newman, Engineering



http://www.veriware.org/autonomous.php



# PRISM: Recent & new developments

- Major new features:
  1. multi-objective model checking
  2. parametric model checking
  3. real-time: probabilistic timed automata (PTAs)
  4. games: stochastic multi-player games (SMGs)

- Further new additions:
  - strategy (adversary) synthesis
  - CTL model checking & counterexample generation
  - enhanced statistical model checking
    (approximations + confidence intervals, acceptance sampling)
  - efficient CTMC model checking
    (fast adaptive uniformisation) [Mateescu et al., CMSB'13]
  - benchmark suite & testing functionality [QEST'12]
    www.prismmodelchecker.org/benchmarks/

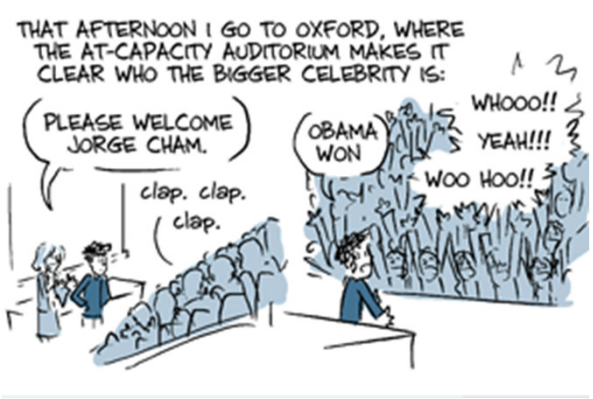


# Summary

- Overview of probabilistic model checking
  - discrete-time Markov chains and Markov decision processes
  - property specifications in temporal logics
  - model checking methods combine graph-theoretic techniques, automata-based methods, numerical equation solving and optimisation
- Ongoing work (not discussed)
  - further models (stochastic games, probabilistic timed/hybrid automata)
  - controller/strategy synthesis
  - runtime verification
  - multiobjective verification and synthesis
  - sampling-based exploration
- Many connections to molecular programming
  - integrate with inference

# PhD Comics and Oxford…










- You are welcome to visit Oxford!
- PhD scholarships, postdocs in verification and synthesis, and more